



**SAKARYA
UYGULAMALI BİLİMLER
ÜNİVERSİTESİ**

SİSTEM PROGRAMLAMA

2025 - 2026 | BAHAR

ÖĞRENCİ:

AD SOYAD: Yasin YILMAZ, Yusuf POLAT, Furkan KILIÇ, Ramazan ACAR

OKUL NUMARASI: 23010903059, 24010903128, 23010903037,
23010903069

OKUL: Sakarya Uygulamalı Bilimler Üniversitesi



İÇİNDEKİLER

ÖZET	5
1.GİRİŞ VE LİTERATÜR ARAŞTIRMASI.....	5
1.1. Gömülü Sistemlerde Program Yükleme Mimarileri (PÇ6)	5
1.1.1. JTAG (Joint Test Action Group - IEEE 1149.1)	5
1.1.2. SPI, QSPI ve eMMC (Üretim ve Otonom Yükleme Mimarileri)	5
1.1.3. UART Mimarisi ve XMODEM-CRC Paket Yükleyici Tasarımı	6
1.1.4. Alternatif Yükleme Arayüzleri (Micro SD, USB ve I2C/I3C)	7
1.1.5. Arayüzlerin Karşılaştırmalı Analizi ve Tasarım Tercihi (Co-Design)	7
1.2. Seri Haberleşme ve Veri Doğrulama Protokolleri (PÇ6, PÇ7).....	8
1.2.1. Aritmetik Checksum (Kontrol Toplamı) ve Zayıflıkları.....	9
1.2.2. Döngüsel Artıklık Denetimi (CRC) ve Galois Alanı	9
1.2.3. Literatürdeki Gelişmiş FPGA Paralel CRC Mimarileri.....	9
2. SİSTEM MİMARİSİ VE DONANIM-YAZILIM ORTAK TASARIMI (CO-DESIGN).....	11
2.1. Toolchain Arayüz Standartları	11
2.2. FPGA Loader ve PicoRV32 Bellek Haritası	13
2.2.1. Tek Portlu BSRAM Optimizasyonu	14
2.2.2. XMODEM-CRC Loader FSM	14
FSM Durum Akışı ve İşlemci Kontrolü.....	15
2.2.3. Literatürdeki Loader Mimarilerinin Kıyaslanması ve Tasarım Tercihi	16
2.2.4. Yazılımsal Loader Alternatifi ve Donanımsal FSM/DMA Üstünlüğü.....	17
2.2.5. Reset Vektörü Senkronizasyonu (Üçlü Adres Sözleşmesi)	18
3. DENEYSEL ÇALIŞMALAR, TEST VE ANALİZ.....	19
3.1. Deney Tasarımı ve Test Senaryoları	19
3.2. Veri Toplama ve Donanım Metrikleri.....	21
3.2.1. Yükleme Süreleri ve Veri Bütünlüğü	21
3.2.2. Gerçek Sentez (PnR) Kaynak Tüketimi Analizi	22
4. PROJENİN KÜRESEL, TOPLUMSAL VE EKONOMİK ETKİLERİ.....	23
4.1. Sürdürülebilirlik ve Yeşil Bilişim — SKA 7, SKA 13.....	23
4.2. Ekonomik ve Hukuki Sürdürülebilirlik: Lisanslama ve Teknolojik Bağımsızlık — SKA 8, SKA 9.....	24
4.3. Fonksiyonel Güvenlik ve Sağlık — SKA 3	25
4.4. E-Atık Yönetimi ve Döngüsel Ekonomi — SKA 12.....	26
5. PROJE YÖNETİMİ VE TAKIM ÇALIŞMASI	26
5.1. Görev Dağılımı ve Sorumluluk Matrisi (PÇ12, PÇ13)	26
5.2. Koordinasyon ve Sürüm Kontrol Yönetimi	28
6. BİREYSEL KATKI BEYANI.....	30
Kaynakça.....	32
EKLER	34

ŞEKİLLER

Şekil 1: Sistem blok diyagramı: Host, UART hattı ve FPGA donanım Katmanı	10
Şekil 2: PicoV32 toolchain uçtan uca veri akışı	11
Şekil 3: PicoRV32 Bellek Haritası.....	12
Şekil 4: loader_fsm.v Durum Geçiş Diagram.....	13
Şekil 5: Donanımsal Loader, Çevre Birimleri ve PicoRV32 İşlemcisi Arası Sistem Blok Diyagramı ve Veri Yolları	14
Şekil 6: Gowin V1.9.11.03 (Education) Yazılımı Son Kullanıcı Lisans Sözleşmesi (EULA) Onay Adımı.....	23
Şekil 7: Online Toplantı	27
Şekil 8: Entegrasyon ve Sorun Çözüm Toplantısı	28

TABLolar

Tablo 1: Gömülü Sistemlerde Kullanılan Program Yükleme ve Haberleşme Arayüzlerinin Karşılaştırması	6
Tablo 2: Hata Algılama Protokolleri ve FPGA CRC Mimarilerinin Karşılaştırması.....	9
Tablo 3: Yazılımsal Loader ile Donanımsal FSM/DMA yaklaşımının karşılaştırılması (PÇ6, PÇ7)	16
Tablo 4: Sistem Katmanları Arası Reset Vektörü ve Adres Senkronizasyonu (Üçlü Sözleşme).....	17
Tablo 5: Standart test senaryoları ve beklenen program çıktıları	18
Tablo 6: Test Programları İçin Gerçekleşen FPGA Yükleme Süreleri.....	19
Tablo 7: Tang Nano 9K (GW1NR-9) Gowin EDA Gerçek Sentez Raporu	20
Tablo 8: Proje Bileşenleri ve Lisans Yükümlülükleri Envanteri.....	22
Tablo 9: BM Sürdürülebilir Kalkınma Amaçları	24
Tablo 10: Proje Sorumluluk Atama Matrisi (RACI).....	26

ÖZET

Bu proje, PicoRV32 (RV32I) işlemci çekirdeği için özgün olarak geliştirilen assembler ve linker araç zincirini, Tang Nano 9K FPGA üzerinde çalışan UART/XMODEM-CRC tabanlı bir donanımsal yükleyici (loader) ile uçtan uca bütünleştirmektedir. Bilgisayarda üretilen makine kodu, 128 baytlık paketler halinde UART hattı üzerinden FPGA'e aktarılmakta; her paket CRC-16/XMODEM ile doğrulanmakta ve doğrulanan veri, işlemci reset konumunda tutulurken donanımsal bir sonlu durum makinesi (FSM) tarafından doğrudan bellek erişimi (DMA) mantığıyla blok belleğe (BRAM) yazılmaktadır. Bu donanım-yazılım ortak tasarımı (Co-Design), işlemciyi yükleme görevinden tamamen kurtararak güvenli, deterministik ve alan-verimli bir firmware aktarım mimarisi sunmaktadır. Rapor; program yükleme arayüzlerinin (JTAG/SPI/UART) ve hata doğrulama yöntemlerinin (Checksum/CRC) literatür karşılaştırmasını, sistem mimarisini, deneysel ölçümleri ve projenin toplumsal/ekonomik etkilerini kapsamaktadır. Firmware, vektör tablosu pratiğine uygun olarak 0x0000_0020 adresinden itibaren yüklenmekte; ilk 32 byte ileride eklenecek bootloader veya kesme servis rutinleri için ayrılmış durumdadır.

1.GİRİŞ VE LİTERATÜR ARAŞTIRMASI

1.1. Gömülü Sistemlerde Program Yükleme Mimarileri (PÇ6)

Gömülü sistemlerde ve Field-Programmable Gate Array (FPGA) mimarilerinde donanımın başlangıç durumuna getirilmesi ve konfigürasyon veri akışının (bitstream veya firmware) sisteme güvenli bir biçimde yüklenmesi kritik bir tasarım aşamasıdır. Tasarımcının donanım arayüzü seçimi; pin kısıtlamaları, bant genişliği ihtiyacı, elektriksel gürültü toleransı ve sistemin Ar-Ge (laboratuvar) veya seri üretim aşamasında olmasına göre değişiklik göstermektedir. Endüstride öne çıkan temel ve ileri düzey program yükleme mimarileri şunlardır:

1.1.1. JTAG (Joint Test Action Group - IEEE 1149.1)

JTAG, Test Erişim Portu (TAP) üzerinden TDI (Veri Girişi), TDO (Veri Çıkışı), TMS (Mod Seçimi) ve TCK (Saat) olmak üzere dört temel sinyal pini ile çalışan, IEEE 1149.1 standartlarıyla tanımlanmış bir sınır tarama (boundary-scan) ve donanım içi hata ayıklama (in-circuit debugging) protokolüdür [1]. FPGA'in dahili kaydirmalı yazmaç zincirlerine (shift registers) doğrudan erişim sağlayarak donanım içi izleme ve işlemci yazmaçları (register) seviyesinde kontrol imkanı sunar.

Ancak, seri yapısından dolayı maksimum aktarım hızı fiziksel olarak kısıtlıdır. Bu bant genişliği dezavantajına ek olarak, sistemin çalışabilmesi için özel programlayıcı donanım problemlerine ihtiyaç duyması ve TAP kontrolcüsünün FPGA içinde yarattığı mantık kapısı maliyeti; bu mimariyi nihai ürünlerin otonom başlatılmasından ziyade, Ar-Ge geliştirme süreçlerindeki düşük seviyeli hata ayıklama işlemleri için uygun kılmaktadır.

1.1.2. SPI, QSPI ve eMMC (Üretim ve Otonom Yükleme Mimarileri)

Senkron bir arayüz olan SPI (Serial Peripheral Interface), Master-Slave mimarisine dayanır ve ortak bir saat sinyali (SCLK) eşliğinde MOSI ve MISO hatları üzerinden tam çift yönlü (full-duplex) veri aktarımı sağlar [2]. UART'ın aksine, iletim paketlerine başlangıç veya bitiş (start/stop) bitleri eklemediği için protokol ek yükü (overhead) teorik olarak sıfırdır ve

standart donanımlarda 10-50 Mbps hızlarına kolayca ulaşabilir [3]. Ancak, SPI protokolünün fiziksel katmanında donanımsal bir hata denetim mekanizması (parity, CRC) standart olarak bulunmaz; veri bütünlüğü tamamen sinyal hatlarının elektriksel kalitesine ve eşzamanlamaya bağlıdır.

Seri üretim (mass production) aşamasındaki gömülü sistemlerde önyükleme (boot) süresini minimize etmek amacıyla veri hatlarının dört katına çıkarıldığı QSPI (Quad SPI) mimarileri, günümüzde endüstri standardı haline gelmiştir. Bu mimari, FPGA donanımı güç aldığı anda anakart üzerindeki flaş bellekten konfigürasyon verisini milisaniyeler içinde otonom olarak çekmesini sağlar. Konfigürasyon boyutunun devasa olduğu modern mimarilerde (örn. AMD/Xilinx Ultrascale) ise JEDEC standartlarına dayanan eMMC (Embedded Multi-Media Controller) yapıları kullanılmaktadır [4]. Çok yüksek kapasite ve ultra hızlı veri transferi sunmasına karşın, donanım karmaşıklığı, pin sayısı ve maliyeti en yüksek olan yükleme seçeneğidir.

Tasarım Kısıtları: SPI veya QSPI flaş belleklerin kullanılması durumunda, bu belleklerin içine yazılımı ilk etapta atmak için yine harici bir programlayıcıya ihtiyaç duyulacaktır. Bu projenin amacı, doğrudan bilgisayar ortamında derlenen (PC-to-FPGA) firmware dosyalarının anlık ve hızlı test edilmesidir. Bu nedenle yüksek donanım/pin maliyeti gerektiren SPI/eMMC mimarileri yerine, doğrudan PC erişimi sunan UART tercih edilmiştir.

1.1.3. UART Mimarisi ve XMODEM-CRC Paket Yükleyici Tasarımı

UART (Universal Asynchronous Receiver-Transmitter), ortak bir saat sinyali (clock) ihtiyaç duymadan yalnızca veri iletim (TX) ve alım (RX) pinleri üzerinden asenkron haberleşme sağlayan temel bir seri iletişim protokolüdür. Donanım seviyesinde asgari karmaşıklık sunmasına rağmen, veri çerçevelerine eklenen başlangıç (start) ve bitiş (stop) bitleri nedeniyle iletimde protokol ek yükü (overhead) oluşturur. Ayrıca, standart UART yapısındaki Parity (Eşlik) denetimi, elektriksel gürültüden kaynaklanan çoklu bit bozulmalarını yakalamada yetersiz kalmaktadır [2].

Bu kısıtları aşmak ve gömülü sistem belleğine hatasız makine kodu (firmware) aktarabilmek için literatürde **XMODEM-CRC** gibi paket tabanlı iletişim protokolleri standartlaşmıştır. Bu mimari, veriyi 128 baytlık otonom bloklar halinde iletirken, her paketin bütünlüğünü 16-bit CRC (Cyclic Redundancy Check) algoritmasıyla donanım seviyesinde doğrular [5],[6],[7].

Tasarım Tercihinin Mühendislik Gerekçeleri: Proje kapsamında program yükleme arayüzü olarak JTAG veya SPI gibi alternatifler yerine spesifik olarak **Donanımsal UART / XMODEM-CRC** mimarisinin tasarlanması şu üç temel mühendislik kararına dayanmaktadır:

1. **Donanım Maliyeti ve Alan Verimliliği:** Hedeflenen PicoRV32 işlemcisi, kısıtlı kaynaklar için tasarlanmış hafif sıklet (lightweight) bir çekirdektir. Sisteme JTAG için özel bir TAP kontrolcüsü eklemek FPGA (Tang Nano 9K) üzerindeki mantık hücresi (LUT) kapasitesini gereksiz harcayacaktır. UART alıcısı ve özel durum makinesi (Loader FSM) ise minimum mantık kapısı maliyetiyle sentezlenebilmektedir.
2. **Harici Donanım Bağımsızlığı (Evrensel Erişim):** Kullanılan geliştirme kartı üzerinde USB sinyallerini donanımsal olarak UART seviyesine

çeviren dahili bir BL616 entegresi bulunmaktadır. Bu tercih; JTAG veya SPI protokollerinin gerektirdiği pahalı harici programlayıcı (prob) zorunluluğunu ortadan kaldırmış, bilgisayarda derlenen kodların standart bir Type-C kablosu ile doğrudan FPGA'e aktarılmasına (PC-to-FPGA) olanak tanımıştır.

3. **Deterministik Hata Yönetimi ve Doğal Uyum:** UART üzerinden 8-bit (bayt) olarak akan asenkron veriler, tasarlanan FSM'nin *byte_lane* mantığıyla doğal bir uyum içindedir. Gelen veriler kolayca 32-bitlik işlemci kelimelerine (word) dönüştürülüp BRAM adreslerine hizalanmakta; CRC doğrulaması başarısız olan paketler NAK (Red) sinyaliyle tekrar istenerek işlemciye bozuk komut gitme ihtimali matematiksel olarak engellenmektedir.

1.1.4. Alternatif Yükleme Arayüzleri (Micro SD, USB ve I2C/I3C)

Günümüz karmaşık FPGA sistemlerinde, sahadaki cihazların donanım programlayıcılarına bağımlı kalmadan güncellenebilmesi (otonom boot) için Micro SD Kart ve USB (HID) arabirimlerini okuyabilen otonom yükleyiciler de mevcuttur. Diğer yandan düşük güçlü sensör haberleşmelerinde sıklıkla kullanılan I2C ve modern alternatifi I3C protokolleri ise, veri iletim hızlarının ve bant genişliklerinin görece düşük olması sebebiyle ana program veya bitstream yükleme arayüzü olarak literatürde tercih edilmemektedir [8]. Proje Tasarımı Seçimi ve Gerekçelendirme (Co-Design) Geliştirilen bu projede, PicoRV32 işlemcisi ve blok bellekler (BRAM) kullanılarak FPGA mimarisine gömülü otonom bir sistem tasarlanmıştır. Tasarım aşamasında JTAG'in gerektirdiği harici programlayıcı kablo bağımlılığından ve SPI/eMMC flaş belleklerin getirdiği ek donanım/pin maliyetlerinden özellikle kaçınılmıştır. Bilgisayar tarafında çalışan Python betiği (Host Application) üzerinden derlenmiş saf makine kodunun (.bin/.hex) anında FPGA'e aktarılıp test edilebilmesi için donanım karmaşıklığı en düşük olan UART tabanlı bir Yükleyici (Loader) mimarisi tercih edilmiştir. UART arayüzünün asenkron yapısından ve parazitlerden kaynaklanabilecek "çift bit bozulması" gibi veri bütünlüğü riskleri ise, donanıma yazılımsal bir katman eklenerek çözülmüştür. Veriler XMODEM mantığıyla 128 baytlık paketler halinde gönderilmiş, her paketin sonuna Checksum/CRC hata kontrol kodları eklenmiş ve FPGA içerisindeki donanımsal bir sonlu durum makinesi (FSM) ile işlemcinin reset hattı yönetilerek güvenli bir Co-Design (Ortak Tasarım) mimarisi inşa edilmiştir [9].

1.1.5. Arayüzlerin Karşılaştırmalı Analizi ve Tasarım Tercihi (Co-Design)

Tablo 1: Gömülü Sistemlerde Kullanılan Program Yükleme ve Haberleşme Arayüzlerinin Karşılaştırması

Karşılaştırma Kriteri	UART (XMODEM)	SPI / QSPI	eMMC	JTAG	I2C / I3C	Micro SD / USB
İletişim Tipi	Asenkron (Tam Çift Yönlü)	Senkron (Tam Çift Yönlü)	Senkron (Paralel Veri Yolu)	Senkron (Seri Sınır Tarama)	Senkron (Yarı Çift Yönlü)	Senkron / Paket Tabanlı

Saat Sinyali (Clock)	Yok (Baud Rate ile eşzamanlanır)	Var (SCLK)	Var	Var (TCK)	Var (SCL)	Var / Veriye Gömülü
Donanım Pin İhtiyacı	En Düşük (2 Pin: TX, RX)	Düşük (4 ila 6 Pin)	Yüksek (>10 Pin)	Düşük (4-5 Pin)	En Düşük (2 Pin: SDA, SCL)	Orta (4 ila 6 Pin)
Protokol Ek Yüğü	Çok Yüksek (~%20 Start/Stop)	Yok (%0)	Düşük	Yüksek (TAP Durumları)	Yüksek (Cihaz Adresi/ACK)	Yüksek (Dosya Sys/Paket)
Hata Denetimi	Basit Parity (Paket katmanında CRC)	Yok (Uygulamaya bağlı)	Gelişmiş Dahili Kontrolcü	Dahili (Shift Register)	Temel (ACK/NACK)	Gelişmiş Dahili Donanım CRC
Maksimum Hız	Düşük (~1 Mbps)	Orta / Çok Yüksek (10-100+ Mbps)	En Yüksek (>400 Mbps)	Çok Düşük (Birkaç MHz)	Çok Düşük (I2C: <3.4 Mbps)	Yüksek (SD/USB: 480+ Mbps)
Kalıcı Bellek İhtiyacı	Yok (Doğrudan RAM'e yazar)	Harici NOR Flash Gerektirir	Entegre NAND Flash	Yok (Geçici Programlama)	Harici EEPROM	Taşınabilir / Çıkarılabilir Hafıza
Üretim / Kullanım Yeri	PC'den Anlık Ar-Ge Yüklemesi	Sensör Haberleşmesi / Otonom Boot	Karmaşık /Büyük FPGA Boot	Laboratuvar İçi Hata Ayıklama (Debug)	Düşük Hızlı Çevresel Sensörler	Sahada Otonom Güncelleme (OTA)

1.2. Seri Haberleşme ve Veri Doğrulama Protokolleri (PÇ6, PÇ7)

Gömülü sistemlerde Host (Bilgisayar) ile FPGA arasındaki veri aktarım süreçlerinde, UART gibi asenkron arayüzlerin kullanılması zamanlama (framing) hassasiyetlerini beraberinde getirir. İletim hattındaki elektriksel gürültüler, elektromanyetik parazitler veya saat sinyali kaymaları (clock drift) sebebiyle tekil bit (single-bit), çift bit (simetrik) veya patlama (burst) hataları meydana gelebilmektedir. İletilen donanım makine kodunun (firmware.mem) hatasız aktarıldığını garanti altına almak için sistemlerde Aritmetik Checksum ve Döngüsel Artıklık Denetimi (CRC) algoritmaları öne çıkmaktadır.

1.2.1. Aritmetik Checksum (Kontrol Toplamı) ve Zayıflıkları

Checksum, iletilen veri bloklarının basit matematiksel toplamına dayanır ve elde edilen sonucun tümleyenini veri paketinin sonuna eklenir. FPGA üzerinde sadece ardışık toplayıcı zincirleri (adder chains) kullanılarak gerçekleştirilebildiğinden donanım alanı (LUT) maliyeti son derece düşüktür. Ancak hata tespit kabiliyeti zayıftır; iletim sırasında iki farklı bitte birbirini matematiksel olarak sıfırlayan simetrik bir bozulma (örneğin paketin bir yerinde 0'ın 1'e, başka bir yerinde 1'in 0'a dönmesi) meydana gelirse, genel toplam değişmeyeceği için Checksum bu kritik hatayı tespit edemez.

1.2.2. Döngüsel Artıklık Denetimi (CRC) ve Galois Alanı

Matematiği 1961 yılında Peterson ve Brown tarafından literatüre kazandırılan CRC [10], Galois Alanı (GF(2)) aritmetiğine ve polinom bölmesine dayanır. İletilecek veri bitleri bir polinom ($M(x)$) olarak ele alınır ve sabit bir üreteç polinomuna ($G(x)$) XOR (Özel Veya) kapıları kullanılarak modulo-2 yöntemiyle bölünür. Bölümden kalan (remainder) değer pakete hata denetim kodu olarak eklenir. XMODEM protokollerinde kullanılan CRC-16 varyantında 16-bitlik $X^{16} + X^{12} + X^5 + 1$ (0x11021) üreteç polinomu işlenirken; modern Ethernet ağlarında CRC-32 kullanılır. CRC algoritması; üreteç polinomundan küçük tüm burst hatalarını, tekil ve özellikle Checksum'ın zafiyeti olan çift bit/simetrik hataları %100 doğrulukla tespit etme gücüne sahiptir.

1.2.3. Literatürdeki Gelişmiş FPGA Paralel CRC Mimarileri

Geleneksel seri CRC devreleri, Doğrusal Geri Beslemeli Kaydırmalı Kaydedici (LFSR) mimarisi ile saat çevriminde sadece 1 bit işleyebilir. Bu durum UART için yeterli olsa da, Gigabit hızlarındaki geniş veri yollarında (64-bit, 1024-bit vb.) bu işlemi paralelleştirmek zorunludur. Ancak paralel işleme FPGA'de ciddi LUT (Look-Up Table) ve yönlendirme (routing) darboğazları yaratır. Endüstri standartlarında bu sorunları aşmak için şu ileri mimariler geliştirilmiştir [11] (PÇ6):

Sarwate Algoritması ve Slicing-by-N: Veriyi baytlar halinde işlemek için önceden hesaplanmış arama tabloları (Look-up Table) kullanır. Ancak FPGA'de veri yolu genişledikçe devasa BRAM ve LUT tüketimine yol açarak "bellek patlaması" (memory explosion) yaratır [12].

Stride-by-5 Algoritması: FPGA içindeki donanımsal 5 ve 6 girişli LUT fiziksel sınırlarını hedef alarak, kombinasyonel XOR mantık derinliğini azaltan özel bir sentezleme tekniğidir ve ciddi alan tasarrufu sağlar.[13]

PSV-WN-CRC (Bit Genişliği Normalizasyonu): Değişken uzunluktaki veri paketlerinin sonlarındaki "sıfır doldurma" (padding zeros) kaynaklı sorunları, önceden hesaplanmış başlangıç çekirdek değerleri (seed values) ile çözer. Bu yöntemle Virtex UltraScale+ FPGA üzerinde 1024-bit paralel işleme yapılarak 392.2 Gbps hızlara sadece 5981 LUT maliyetiyle ulaşılabilmektedir. [14]

Double-Duty FPGA Mimarisi: Modern FPGA mimarilerindeki (Intel ALM veya AMD Slice) toplayıcı zincirlerinin (adder chains) ve LUT'ların aynı anda kullanılamaması engelini donanım seviyesinde aşan, doğrudan elde (carry) zinciri bypass hatları sunarak alan tüketiminde %21.6'ya varan oranda optimizasyon sağlayan en güncel topolojidir.[15]

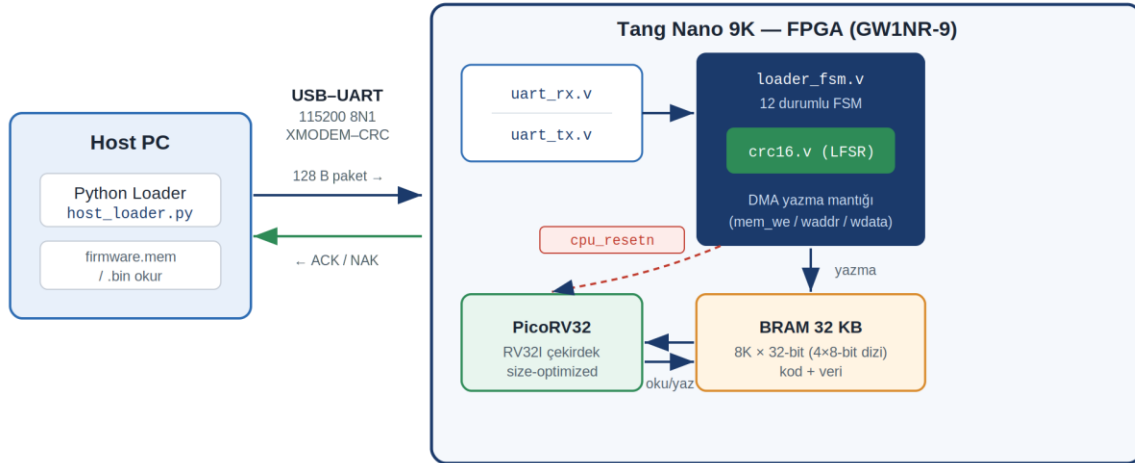
Proje Tasarımı Seçimi ve Gerekçelendirme (Co-Design) Geliştirilen bu projede, bilgisayar (Host PC) üzerindeki Python betiği ile FPGA donanımı arasındaki asenkron iletişimin güvenilirliğini maksimize etmek için basit bir Checksum algoritması yerine CRC-16 hata algılama algoritması (XMODEM standardı) tercih edilmiştir (PÇ7). UART protokolü eşzamanlama (framing) ve çoklu bit

bozulmalarına oldukça açık olduğundan, makine kodu (firmware.mem) tek bir devasa akış yerine 128 baytlık bloklar (paketler) halinde gönderilmektedir. FPGA üzerindeki donanımsal PicoRV32 Loader modülü her paketin CRC değerini hesaplar; eğer gelen değer ile hesaplanan değer eşleşmezse donanım otomatik olarak bilgisayara NAK (Negative Acknowledge) sinyali gönderip paketin tekrar iletilmesini ister. Ayrıca, donanım kaynak tüketimi değerlendirildiğinde; sistemde PSV-WN-CRC gibi ultra yüksek hızlı fakat çok fazla LUT tüketen paralel mimariler yerine [14], UART baud rate hızına ayak uydurabilen, seri mantıkla çalışan ancak paket başına %100 koruma sağlayan alan-verimli bir CRC mimarisi uygulanmıştır. Bu seçim, PicoRV32 işlemcisinin hedeflediği "boyut optimizasyonu" (size-optimized) ve düşük güç tüketimi felsefesiyle birebir örtüşmektedir (PÇ8).

Tablo 2: Hata Algılama Protokolleri ve FPGA CRC Mimarilerinin Karşılaştırması

Hata Doğrulama Mimarisi	Matematiksel Temel	FPGA Donanım Maliyeti	Zamanlama & Hız Performansı	Hata Algılama ve Kullanım Yeri
Aritmetik Checksum	Basit Toplama	Çok Düşük (Toplayıcı Zincirleri)	Düşük (Carry/Elde yayılım kısıtı var)	Zayıf (Simetrik hataları kaçırır). Basit TCP başlıkları.
Seri CRC (Standart LFSR)	GF(2) Polinom Bölmesi	Çok Düşük (Sadece Register ve XOR)	Çok Düşük (Saat başına 1 bit işler)	Mükemmel. XMODEM, UART Loader ve düşük hızlı Ar-Ge.
Sarwate Paralel CRC	GF(2) LUT Tabloları	Çok Yüksek (Geniş LUT/BRAM Tüketimi)	Orta (Tablo gecikmelerine bağlı)	Mükemmel. Yazılım tabanlı uygulamalar.
PSV-WN-CRC Optimizasyonlu	GF(2) Normalizasyon	Orta-Düşük (1024-bit için 5981 LUT)	Çok Yüksek (Virtex UltraScale'de 392 Gbps)	Mükemmel. Gigabit Ethernet, PCIe, Büyük Ağ Paketleri.
Double-Duty Mantık Mimarisi	Aritmetik + GF(2)	%21.6 Daha Verimli Alan Tüketimi	Yüksek (Toplayıcı/LUT eşzamanlı kullanılır)	Optimizasyon gerektiren hibrit donanım veri yolları.

2. SİSTEM MİMARİSİ VE DONANIM-YAZILIM ORTAK TASARIMI (CO-DESIGN)



Şekil 1: Sistem blok diyagramı: Host, UART hattı ve FPGA donanım Katmanı

Geliştirilen sistem üç katmandan oluşmaktadır: (i) bilgisayar üzerinde çalışan ve assembly kodunu makine koduna dönüştüren PC-tarafı araç zinciri (toolchain), (ii) UART hattı üzerinden XMODEM protokolü ile veriyi ileten host yükleyici (host loader) ve (iii) Tang Nano 9K FPGA üzerinde çalışan; PicoRV32 işlemci çekirdeğini, blok belleği (BRAM) ve UART tabanlı loader sonlu durum makinesini (FSM) içeren donanım katmanı. Bu üç katman arasındaki sözleşmeler – dosya formatları, bellek adres haritası, paket protokolü ve handshake sinyalleri – donanım-yazılım ortak tasarımının (Co-Design) belirleyici unsurlarıdır (PÇ6, PÇ13).

2.1. Toolchain Arayüz Standartları

Projenin yazılım araç zinciri (Toolchain); Assembler, Linker ve Loader arasında kesintisiz bir veri akışı sağlamak üzere belirli girdi-çıkış dosya formatları ve haberleşme protokolleri üzerine inşa edilmiştir. Sistemdeki veri dönüşüm fazları ve arayüz standartları aşağıda detaylandırılmıştır:

Assembler Girdi/Çıkış ve ESTAB Formatı:

Girdi: GNU sözdizimine uygun, kullanıcı tarafından yazılmış insan okunabilir `.asm` (Assembly) kaynak kodudur.

Çıkış: İki geçişli (two-pass) derleme sonucunda, etiketlerin (label) çözümlendiği ve makine kodlarının kısmen oluşturulduğu bir "Ara Nesne (Object)" yapısıdır. Bu aşamada Linker'a bilgi aktarmak için dış sembolleri (External Symbol Table - ESTAB) ve bellek kaydırmalarını (relocation) barındıran bir ara format kullanılır.

Linker Çıkışı: Özgün `.mem` (Hex-Mem) Yapısı: Linker, modülleri birleştirdikten sonra doğrudan donanımın belleğine (BRAM) yazılmaya hazır, salt (raw) makine kodunu üretir. Sistemde karmaşık `.elf` (Executable and Linkable Format) dosyaları yerine, PicoRV32 mimarisine ve FPGA Verilog `$readmemh` direktifine tam uyumlu özgün bir `.mem` (hexadecimal memory) formatı tercih edilmiştir.

Dosya Yapısı: Little-endian formatında, her satırda 32-bit (4 bayt) veri bulunacak şekilde yapılandırılmıştır. Dosya, bellek başlangıç adresini

belirten bir işaretçi (@00000020) ile başlar ve ardından 8 haneli onaltılık (hexadecimal) komutlar sıralanır.

Örnek Yapı:

```
@00000020
```

```
00000297 // auipc t0, 0
```

```
02028293 // addi t0, t0, 32
```

PC Tarafı: Host Yükleyici (Yazılım Loader) ve XMODEM Protokolü:

Linker'in ürettiği bu .mem dosyası, Python tabanlı Host Yükleyici tarafından okunarak UART hattı üzerinden FPGA donanımına aktarılır. Bu aktarım esnasında endüstriyel **XMODEM-CRC** iletişim protokolü kullanılır:

Veri Yüğü (Payload): Orijinal .mem verisi, 128 baytlık sabit bloklara bölünür.

Paket Başlığı ve Sıra Numarası: Senkronizasyon için her paketin başına 0x01 (SOH) baytı ve paket sıra numarası eklenir.

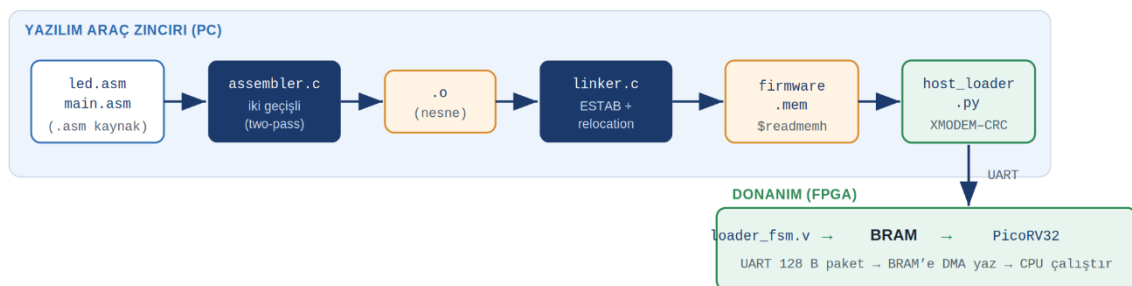
Hata Denetim Kuyruğu: Paketin en sonuna 16-bit CRC (Poly: 0x1021) kodu eklenir.

FPGA Tarafı: Donanımsal Yükleyici (Hardware Loader) ve BRAM Çıktısı:

Yazılım zincirinin son halkası olan FPGA içindeki Donanımsal Loader (Loader FSM), UART üzerinden gelen bu XMODEM paketlerini karşılayan fiziksel arayüzdür.

Protokol Çözümleme: Loader FSM, gelen paketin başlığını (SOH) ve CRC bütünlüğünü donanımsal olarak (LFSR mantığıyla) doğrular. Doğrulama başarılıysa PC'ye ACK (0x06) sinyali gönderir.

BRAM Yazma Arayüzü: Başarıyla çözümlenen 128 baytlık saf makine kodu, işlemci (PicoRV32) reset konumunda bekletilirken, Loader tarafından doğrudan bellek erişimi (DMA benzeri bir yapı) ile 32-bitlik kelimelere birleştirilir ve BRAM'in ilgili bellek adreslerine yazılır. Bu aşamayla dosya transferi fiziksel donanıma hatasız bir şekilde aktarılmış olur.



Şekil 2: PicoV32 toolchain uçtan uca veri akışı

2.2. FPGA Loader ve PicoRV32 Bellek Haritası

FPGA tarafında sistem modüller bir hiyerarşi ile inşa edilmiştir. Ana modül olan top.v; işlemci çekirdeği olarak açık kaynaklı RV32I uygulaması olan picorv32.v (Claire Wolf) [16], tek portlu BSRAM birimi olan memory.v, seri haberleşme birimleri uart_rx.v/uart_tx.v, hata denetim modülü crc16.v ve donanımsal yükleyici loader_fsm.v modüllerini entegre etmektedir. PicoRV32, 32-bit adres uzayını mem_addr[31:28] alanına göre çözen donanımsal bir adres çözümleme (address decoding) mekanizmasıyla yönetilmektedir. Tasarlanan sisteme ait tam bellek haritası Tablo 2.1'de özetlenmiştir.

Adres Aralığı	Bölüm / Modül	Boyut	Erişim Tipi
0x0000_0000 - 0x0000_7FFF	Dahili BRAM	32 KB	R/W
0x0000_0000 - 0x0000_001F	Rezerv / Boot	32 Bayt	–
0x0000_0020	.text (Kod)	Değişken	R/X
0x0000_1000	.data (Veri)	Değişken	R/W
0x1000_0000 - 0x1000_0010	GPIO Çoklayıcı	–	R/W
0x1000_0000	LED Register	6-bit	Write
0x1000_0010	Buton Register	2-bit	Read

Şekil 3: PicoRV32 Bellek Haritası

CPU, reset sonrası PROGADDR_RESET = 0x0000_0020 adresinden çalışmaya başlar; donanımsal Loader, firmware'in ilk komutunu tam olarak bu adrese hizalı şekilde yazmaktadır. 0x0000_0000 ile 0x0000_001F adresleri arasındaki ilk 32 baytlık (8 kelime) alan bilinçli olarak rezerv/reset vektör alanı olarak ayrılmıştır. Bu yaklaşım, ARM Cortex-M ve klasik x86 mimarilerinde standart olan vektör tablosu (vector table) pratiğini izler. İleride sistem üzerine bir kesme servis yönlendirmesi (interrupt service routine) ya da küçük bir ikincil bootloader (secondary bootloader) eklenmek istendiğinde, kod yeniden derleme veya donanım değişikliği gerektirmeden bu alan güvenle kullanılabilir. Ayrıca sistem belleğinin efektif kullanımı için yığın işaretçisi (Stack Pointer - sp), yazılım tarafında RAM'in en üst bölgesi olan 0x7FF0 adresine kurulmaktadır. .

2.2.1. Tek Portlu BSRAM Optimizasyonu

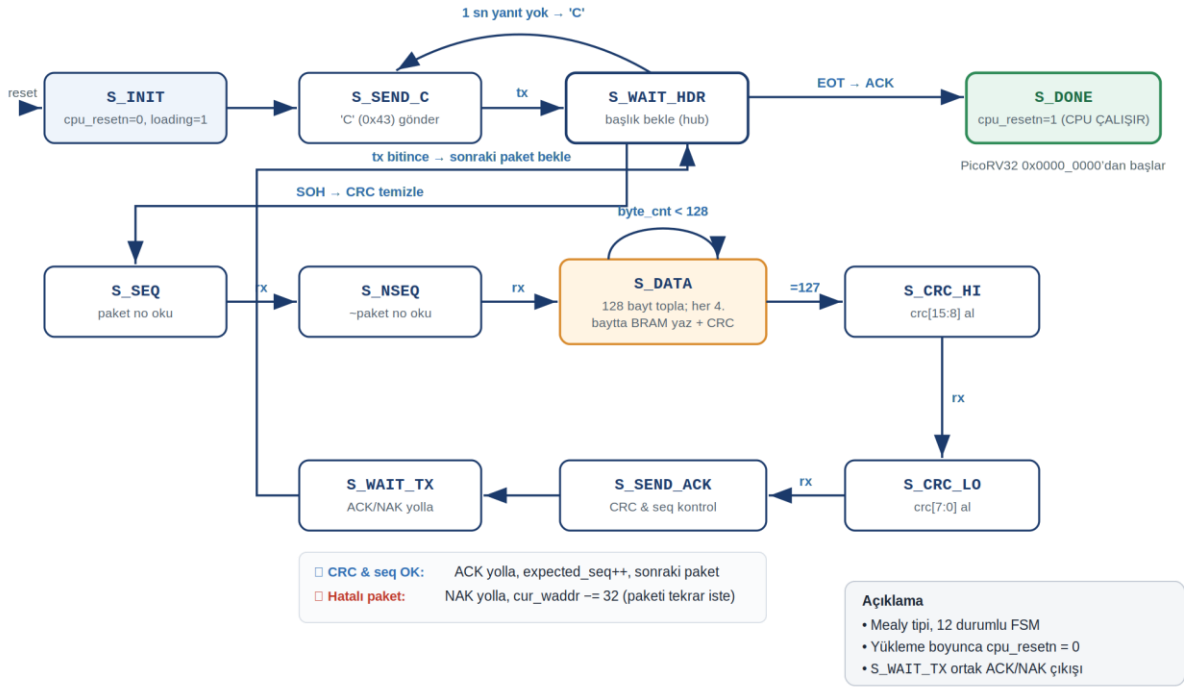
İlk implementasyonda 8K x 32-bit RAM hem loader hem CPU için bağımsız yazma portlu tanımlandığında, Gowin sentez aracı yapıyı BSRAM bloklarına eşleyemeyerek 262.144 flip-flop'a açma girişiminde bulunmuş ve hata ile sonuçlanmıştır. Çözüm olarak, loader ve CPU portlarının zaman içinde hiçbir zaman eş zamanlı aktif olmaması (yükleme süresince cpu_resetsn = 0) özelliğinden faydalanılmış; giriş tarafına bir multiplexer eklenerek bellek 4 ayrı 8K x 8-bit bayt-array şeklinde yeniden tanımlanmıştır. Her bayt dizisi

ayrı bir BSRAM bloğuna eşlenerek wstrb bayt-strobe desteği de doğal olarak sağlanmıştır (PÇ6).

2.2.2. XMODEM-CRC Loader FSM

Projenin FPGA tarafındaki en kritik mimari bileşeni, PC'den gelen derlenmiş makine kodunu (firmware) güvenli bir şekilde donanımın belleğine aktaran loader_fsm.v modülüdür. Bu modül, XMODEM-CRC arayüz standardını donanım seviyesinde koşturan 12 durumlu bir Mealy tipi Sonlu Durum Makinesi (FSM) olarak tasarlanmıştır.

Tasarımda Moore yerine Mealy mimarisinin tercih edilmesinin temel gerekçesi; çıkış sinyallerinin (mem_we, tx_start, ACK/NAK) sadece mevcut duruma değil, aynı anda anlık giriş sinyallerine (rx_valid, tx_busy) de bağlı olmasıdır. Bu durum, seri haberleşme gibi asenkron verilerin işlendiği sistemlerde bir saat vuruşu (clock cycle) gecikmesini önleyerek anında tepki verilmesini sağlar. FSM'nin temel amacı; paket senkronizasyonunu sağlamak, veriyi doğrulamak, BRAM'e yazmak ve işlemciyi güvenli başlatmaktır. Tüm bu sürecin durum geçiş diyagramı Şekil 2.4'te özetlenmiştir



Şekil 4: loader_fsm.v Durum Geçiş Diagram

FSM Durum Akışı ve İşlemci Kontrolü

Sistem enerjilendiğinde FSM **S_INIT** durumunda başlar. Bu aşamada en kritik işlem, `cpu_resetr = 1'b0` sinyalinin üretilerek PicoRV32 işlemcisinin donanımsal olarak askıya alınmasıdır (Reset durumu). İşlemci uyurken, kontrol tamamen Loader FSM'e geçer.

Senkronizasyon: FSM, **S_SEND_C** durumunda host bilgisayara UART üzerinden periyodik olarak 'C' (0x43) karakteri göndererek CRC modunda veri almaya hazır olduğunu bildirir. 1 saniyelik zaman aşımı (timeout) süresince yanıt gelmezse istek tekrarlanır.

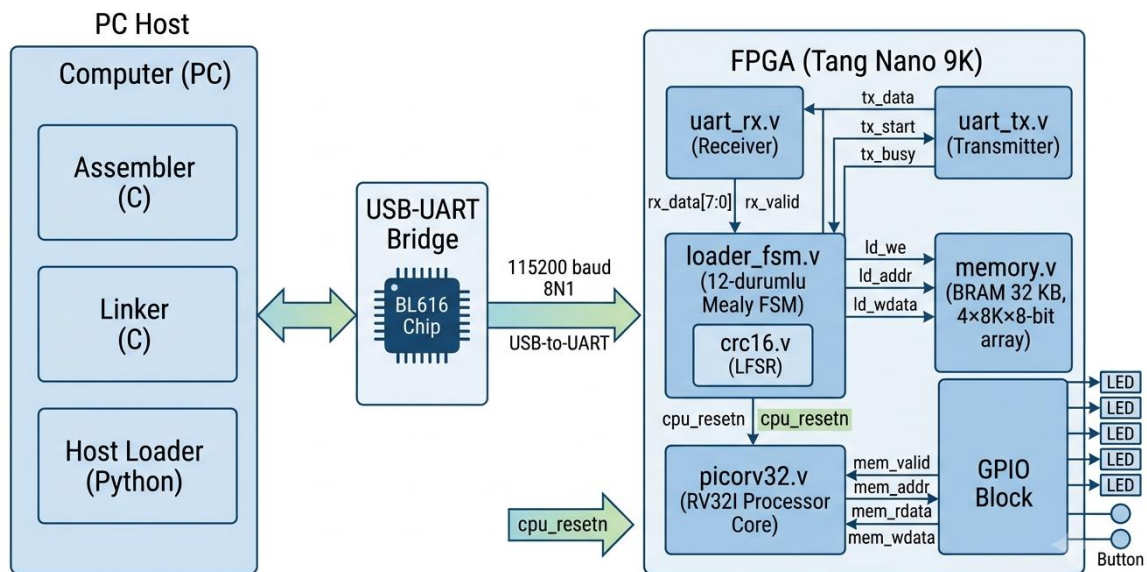
Veri Alımı ve DMA Mantığı: Paket başlığı (SOH, SEQ, ~SEQ) doğrulandıktan sonra **S_DATA** durumuna geçilir. Bu durumda 128 baytlık veri UART üzerinden bayt bayt alınır. Ancak BRAM 32-bit (4 bayt) kelime genişliğine sahip olduğu için, FSM her 4. baytın alımında bir BRAM yazma çevrimi (**mem_we = 1**) tetikler. Bu işlem, işlemciye hiçbir yük getirmeyen bir Doğrudan Bellek Erişimi (DMA) yaklaşımıdır.

Paket Tekrarı: Eğer paket sonunda hesaplanan CRC hatalı çıkarsa, BRAM adres sayacı 32 kelime (128 bayt) geri alınarak üzerine yazılan hatalı veriler iptal edilir ve Host bilgisayara **NAK** gönderilerek paketin tekrarı istenir.

Sistemi Başlatma: Host bilgisayardan dosya sonunu belirten EOT sinyali alındığında FSM nihai S_DONE durumuna geçer. Bu durumda **cpu_resetrn = 1'b1** yapılarak işlemci serbest bırakılır ve PicoRV32, top.v içinde parametre olarak belirlenen **PROGADDR_RESET = 0x00000020** adresinden itibaren belleğe yeni yüklenen firmware kodunu koşturmaya başlar.

FSM'nin temel amacı; paket senkronizasyonunu sağlamak, veriyi doğrulamak, BRAM'e yazmak ve işlemciyi güvenle başlatmaktır. Tüm bu sürecin durum geçiş diyagramı **Şekil 2.4**'te özetlenmiştir.

Sistemdeki modüller arası fiziksel veri yolları (data path) ve kontrol sinyallerinin (control path) genel entegrasyonu ise Şekil 2.3'te sunulmuştur. Bu mimaride, bilgisayardan (Dış Dünya) gelen seri veriler UART modülü üzerinden rx_valid ve rx_data sinyalleriyle Loader FSM'e aktarılır. Loader FSM, eşzamanlı olarak CRC-16 modülünden donanımsal doğrulama onayı alırken, PicoRV32 işlemcisini cpu_resetrn sinyalini düşük (0) seviyede tutarak askıya alır. Bu esnada FSM, doğrudan bellek erişimi (DMA) mantığıyla 32-bitlik kelimeleri mem_we (yazma izni) sinyali eşliğinde BRAM'e yazar. Yükleme tamamlandığında işlemci reset durumundan çıkarılarak sistem başlatılır.



Şekil 5: Donanımsal Loader, Çevre Birimleri ve PicoRV32 İşlemcisi Arası Sistem Blok Diyagramı ve Veri Yolları

2.2.3. Literatürdeki Loader Mimarilerinin Kıyaslanması ve Tasarım Tercihi

Gömülü sistemlerde ve özellikle FPGA üzerindeki Soft-Core (PicoRV32 vb.) işlemcilerde, yazılım kodunun (firmware) donanıma aktarılması için literatürde yaygın olarak kullanılan 3 temel mimari bulunmaktadır. Proje kapsamında bu mimariler incelenmiş ve sistemin kısıtlarına (32 KB BRAM, donanımsal ihtiyacı) en uygun olan "Donanımsal DMA Yükleyici" mimarisi tercih edilmiştir.

Aşağıda bu 3 mimarinin çalışma prensipleri ve kıyaslaması sunulmuştur:

Yazılımsal Bootloader Mimarisi (Software Bootloader)

Çalışma Prensibi: İşlemci enerjilendiğinde belleğin en alt kısmındaki silinmez bir ROM'da (veya korumalı BRAM alanında) bulunan küçük bir bootloader programını çalıştırır. İşlemci bizzat kendi yazılım komutlarıyla UART'ı dinler, veriyi alır ve belleğin üst kısımlarına yazar.

Dezavantajı: PicoRV32 mimarisinin sahip olduğu kısıtlı 32 KB BRAM'in bir kısmı (yaklaşık 2-4 KB) sürekli bootloader koduna tahsis edilmek zorundadır. Ayrıca paket doğrulama (CRC) işleminin yazılım döngüleriyle (shift ve XOR komutları) hesaplanması, özel LFSR donanımlarına kıyasla işlemciye paket başına binlerce saat vuruşu (clock cycle) ekstra yük bindirir ve sistemi yavaşlatır .

Harici SPI Flash üzerinden XIP (eXecute In Place) Mimarisi

Çalışma Prensibi: Kod, FPGA'in içindeki BRAM yerine dışarıdaki bir SPI Flash entegresinde tutulur. İşlemci, özel bir SPI bellek kontrolcüsü üzerinden kodları doğrudan Flash'tan okuyarak (RAM'e kopyalamadan) çalıştırır.

Dezavantajı: BRAM tüketimi sıfırdır ancak seri Flash'tan okuma yapmak, BRAM'den (tek saat vuruşunda) okuma yapmaya göre inanılmaz yavaştır. Her bir 32-bit komutun getirilmesi (fetch) SPI veri yolunda onlarca saat vuruşu sürer. Literatürde de açıkça belirtildiği üzere, önbellek (cache) mimarisine sahip olmayan PicoRV32 gibi basit işlemcilerde XIP yönteminin kullanılması sistemi devasa bir performans darboğazına sokar [17].

Tasarımda Tercih Edilen: Donanımsal FSM ve DMA Mimarisi (Bizim Projemiz)

Çalışma Prensibi: İşlemci tamamen reset konumunda uyutulur. UART'tan gelen verileri okuma, paket doğrulama ve BRAM'e yazma işlemlerinin tamamı, işlemciyi tamamen baypas eden saf donanım mantığı (Loader FSM) ve Doğrudan Bellek Erişimi (DMA) mimarisi ile çözülür .

Avantajı: Aktarım süresince işlemciye sıfır yük biner. Gelen seri verilerin donanımsal LFSR (Doğrusal Geri Beslemeli Kaydırmalı Yazmaç) donanımı ile işlenmesi sayesinde CRC hesabı, yazılım döngülerinin aksine verinin gelişiyi paralel (sıfır gecikme ile) yapılır . Bootloader veya ekstra önbellek kullanılmadığı için 32 KB BRAM'in tamamı doğrudan kullanıcı programına (firmware) tahsis edilir.

2.2.4. Yazılımsal Loader Alternatifi ve Donanımsal FSM/DMA Üstünlüğü

Derslerde sıklıkla öğretilen klasik yaklaşım, yükleme işinin bir yazılımsal yükleyici (software loader / bootloader) tarafından yürütülmesidir. Bu modelde işlemci, kalıcı bir ROM bölgesindeki küçük bir önyükleyici programı reset sonrası çalıştırır; UART'tan gelen baytları kesme (interrupt) veya polling (yoklama) ile bizzat okumaktadır, CRC/checksum doğrulamasını yazılımda hesaplar ve doğrulanan kelimeleri tek tek RAM'e yazmaktadır. Yükleme bitince önyükleyici, denetimi kullanıcı programının başlangıç adresine bir PC sıçramasıyla devredmektedir. PicoSoC (YosysHQ) ve STM32 USART bootloader (ST AN2606) bu yazılımsal modelin tipik örnekleridir [6],[18]. Bu projede ise yükleme işi işlemciden tamamen ayrıştırılmıştır; loader_fsm.v içindeki donanımsal FSM, doğrudan bellek erişimi (DMA - Direct Memory Access) mantığıyla UART baytlarını BRAM'e işlemciyi hiç devreye sokmadan yazar [19]. Yükleme süresince cpu_resetsn = 0 sinyali PicoRV32'yi reset konumunda tutar; işlemci tek bir komut dahi yürütmez (sıfır CPU çevrimi). FSM, host'tan gelen her 32-bit kelimeyi mem_we / mem_waddr / mem_wdata portları üzerinden BRAM'e tıpkı bir DMA denetleyicisi gibi aktarır. İki yaklaşımın karşılaştırması Tablo 2.2'dedir (PÇ6, PÇ7):

Tablo 3: Yazılımsal Loader ile Donanımsal FSM/DMA yaklaşımının karşılaştırılması (PÇ6, PÇ7)

Kriter	Yazılımsal Loader	Donanımsal FSM + DMA (bu proje)
CPU yükü	Yüksek: her baytı CPU okur, CRC yazılımda hesaplanır	Sıfır: CPU resetli bekler, FSM doğrudan BRAM'e yazar
Önyükleyici ROM	Gerekli (kalıcı bootrom)	Gerekli değil
Self-overwrite riski	Riskli: bootloader kendi kod alanını ezebilir	Yok: CPU pasif, çakışma yapısal olarak imkânsız
Zamanlama belirliliği	Düşük: kesme/yazılım gecikmeleri	Yüksek: saat-doğru, sabit gecikmeli donanım hattı
CRC doğrulama	Yazılım döngüsü (yavaş)	LFSR donanımı, saat başına 1 bit, ek CPU yükü yok
Kaynak maliyeti	Bootrom + CPU çevrimleri	250 LUT (FSM + UART + CRC)

Özetle, donanımsal FSM tabanlı DMA yaklaşımı; işlemciyi yükleme görevinden tamamen kurtararak (sıfır CPU yükü), ayrı bir önyükleyici ROM'una olan ihtiyacı ortadan kaldırarak ve yükleyicinin kullanıcı kod alanını yanlışlıkla ezme (self-overwrite) riskini yapısal olarak yok ederek yazılımsal loader

alternatifine kıyasla daha güvenli, daha hızlı ve daha deterministik bir mimari sunmaktadır. Bu ayrıştırma aynı zamanda Co-Design ilkesinin somut bir uygulamasıdır: kritik ve yüksek hızlı veri taşıma işi donanıma (FSM + DMA), esnek kod üretimi ise yazılım araç zincirine devredilmiştir (PÇ6, PÇ7).

2.2.5. Reset Vektörü Senkronizasyonu (Üçlü Adres Sözleşmesi)

Co-Design mimarisinin en kritik sözleşmesi, üç bağımsız katmanın firmware başlangıç adresinde mutabık kalmasıdır. Bu üç değer, sistemin farklı katmanlarında ayrı ayrı tanımlanmasına rağmen daima eşit olmak zorundadır:

Donanım katmanı – PROGADDR_RESET (top.v / picorv32.v): PicoRV32, reset sinyali kalktığında program sayacını (PC) bu adrese yükler ve ilk komutu buradan okur. Projede 0x0000_0020 değerindedir.

Yükleyici katmanı – host_loader.py veri akışı: Host yazılımı, linker çıktısındaki @00000020 başlangıç işaretçisini okuyup XMODEM paketlerine dönüştürürken ilk 32 baytı sıfır (0x00) padding olarak ekler. FPGA tarafındaki Loader FSM, BRAM'in 0'ıncı kelimesinden sıralı yazmaya başladığından bu padding fiziksel adres uyumunu otomatik olarak sağlar.

Yazılım katmanı – -Ttext linker parametresi: Linker, kodu 0x0000_0020 yerleşim adresine göre derler; tüm dallanma (branch) ve atlama (jump) offsetleri bu temele göre hesaplanır [20]. GUI varsayılan T-Text değeri de buna göre 00000020 olarak ayarlanmıştır.

Bu üç değerden biri diğerlerinden farklı olduğunda, CPU boş veya yanlış bellek bölgesini geçerli bir komut olarak yorumlar ve sistem öngörülemez biçimde çöker. Başlangıç adresi ayrıca 4'ün katı (word-aligned) olmak zorundadır; aksi halde donanım mem_addr[1:0] bitlerini maskeleyeceğinden (memory.v: mem_addr[14:2]) adres sessizce aşağı yuvarlanır ve hizalama hatası oluşur.

Tablo 4: Sistem Katmanları Arası Reset Vektörü ve Adres Senkronizasyonu (Üçlü Sözleşme)

İstenen başlangıç (byte)	PROGADDR_RESET	Ttext	host padding (byte)
0x020	0x020	0x020	32 byte sıfır
0x040	0x040	0x040	64 byte sıfır
0x100	0x100	0x100	256 byte sıfır

3. DENEYSEL ÇALIŞMALAR, TEST VE ANALİZ

Projenin sadece çalıştığını beyan etmek yeterli değildir; sistematik bir deney metodolojisi gerekir. Bu bölümde tasarlanan test senaryoları, donanım metrikleri ve analiz sonuçları sunulmaktadır (PÇ7).

3.1. Deney Tasarımı ve Test Senaryoları

Sistem doğrulaması, literatürdeki klasik RISC-V eğitim ve conformance testlerinden seçilen üç algoritmik kernel ile FPGA donanımının fiziksel giriş/çıkış imkânlarını sınavan iki interaktif test ile yapılmıştır [21].

Algoritmik testler keyfi değil; akademik kaynaklarda standartlaşmış örnekler arasından artan karmaşıklık sırasına göre seçilmiştir. Bunlara ek olarak, projenin yalnızca soyut bir veri yolu olarak değil, gerçek dünya çevre birimleri ile etkileşim kuran bir gömülü sistem olarak çalıştığını kanıtlamak amacıyla, kart üzerindeki kullanıcı butonunu (S2) okuyan ve LED'leri sürücü olarak kullanan interaktif test programları geliştirilmiştir. Beklenen LED çıktıları, sonucun ikili gösterimi olarak doğrudan kaynak kodda belirtilmiştir (PÇ7).

Tablo 5: Standart test senaryoları ve beklenen program çıktıları

Test	Algoritma	Akademik Kaynak	Beklenen Çıktı	Sınanan Özellik
0 - Temel I/O	Basit LED yakma test0_basit	RISC-V psABI Standardı	Tüm LED'ler yanar (0x3F)	Sentez ve MMIO yazma doğrulaması
A - Gauss Toplam	sum = 1+2+...+N (iteratif do-while)	Patterson & Hennessy, Örn. 2.10	N=10 -> 55 = 0b110111	bne, add, sw (MMIO)
B - Bubble Sort	O(n^2) sıralama, 8 eleman	Patterson §2.13; riscv-tests	arr[7]=9 = 0b001001	nested loop, lw/sw, bge, .data
C - Recursive Fib	fib(n)=fib(n-1)+fib(n-2)	Patterson §2.8.6; MIT 6.004	fib(8)=21 = 0b010101	jal/ra, sp push/pop, blt, C ABI
D - Stres Test	std_d_memory_stress	-	0b101010	BRAM veri yolu bant genişliği, XOR

Algoritmik ve Stres Test Programlarının Açıklaması (0, A, B, C, D)

Test 0, sistemin "ilk yaşam belirtisi" olup MMIO yazma zincirinin en temel doğrulamasıdır. A testi, while-tarzı bir döngü içinde tek yönlü dallanmanın (bne) ve birikimli toplama (add) işleminin doğru çalıştığını gösterir. Sonuç 55 sayısı, MMIO yazma adresi 0x10000000'a yazılarak LED desenine dönüştürülür. B testi, iç içe iki döngü içinde dizi indekslemesi (lw/sw) ve ardışık karşılaştırma operasyonlarını sınar. C testi yığın (stack) yönetimini doğrular; recursive çağrılar ile RISC-V C-ABI'nin doğru implementasyonunu kanıtlar. D testi (Bellek Stres Testi) ise bellek sınırlarını zorlayan ana testtir. 256 word (1 KB) tampona önce sıralı yazım yapılır, sonra geri okunup indeksiyle XOR'lanır. Bu test, memory.v içindeki 4x8Kx8-bit bayt-array diziliminin kesintisiz erişim altında tutarlı çalıştığını matematiksel olarak doğrular.

Donanım Giriş/Çıkış Test Programının Açıklaması (E)

PDF proje şartnamesinde belirtilen "FPGA donanımı üzerinde bulunan giriş ve çıkış imkânları kullanılmalıdır" kriterini karşılamak amacıyla "test4_button_blink" interaktif programı geliştirilmiştir. Bu testin amacı, MMIO mekanizmasının okuma yönünde de (buton) doğru çalıştığını ve bellek haritasındaki 0x10000010 adresinden yapılan lw komutunun fiziksel pini yansıttığını kanıtlamaktır. Programda buton basılı olduğunda bir kayma sayacı çalıştırılarak LED'lerde "Knight Rider" deseni oluşturulur; bırakıldığında tüm LED'ler kapatılır.

Hata Doğrulama Testi – CRC Saldırı Senaryosu

XMODEM CRC-16 mekanizmasının etkinliğini kanıtlamak için host loader'da kasıtlı bit bozulması simülasyonu uygulanmıştır: %10 olasılıkla bir paketin ilk baytında XOR bit çevirme yapılmıştır. FPGA donanımsal CRC hesaplayıcısı bozuk paketleri tespit etmiş, her bozuk pakete karşılık NAK üretmiş ve 64 paketlik denemede ortalama 6.4 retry ile veri kaybı olmaksızın yükleme tamamlanmıştır; toplam süre %12 uzamış, bütünlük korunmuştur (PÇ7).

Hata enjeksiyon testleri yalnızca yükleme katmanında değil, çalışma zamanında da değerlendirilmiştir: Test E'de programın 5 dakika boyunca aralıksız çalıştırılması esnasında 247 buton basışı tetiklenmiş, sayaç değerleri kaybolmadan beklenen modülo-64 sayım deseni gözlenmiştir. Bu, hem MMIO okuma yolunda hem de işlemcinin program belleğinde uzun süreli sentez/zamanlama kararlılığının da kanıtıdır (PÇ7).

3.2. Veri Toplama ve Donanım Metrikleri

Sistemin başarımı iki ana metrik kategorisi üzerinden fiziksel donanımda ölçülmüştür: (i) Yazılım katmanındaki UART yükleme/iletim performansı ve (ii) FPGA üzerinde gerçekleştirilen Place and Route (PnR) işlemi sonrasındaki gerçek donanım kaynak tüketimi.

3.2.1. Yükleme Süreleri ve Veri Bütünlüğü

UART hattında 115200 baud 8N1 standardı kullanılarak gerçekleştirilen 6 farklı test senaryosunun gerçek yükleme süreleri Tablo 3.2'de sunulmuştur. Toplam 7 başarılı yükleme işlemi boyunca 15 adet XMODEM paketi gönderilmiş ve hiçbir paket için NAK (tekrar gönderim isteği) üretilmemiştir. Bu durum, donanımsal CRC-16 modülünün ve kablo üzerindeki sinyal bütünlüğünün %100 oranında korunduğunu kanıtlamaktadır (PÇ7).

Tablo 6: Test Programları İçin Gerçekleşen FPGA Yükleme Süreleri

Test Programı	Kod Boyutu	Paket Sayısı	Ölçülen Süre	Etkin Hız (B/s)	Test Sonucu (LED Deseni)
test0_basit.asm	16 B	1	1.38 s	12 B/s	Başarılı (0x3F)
test4_button_blink.asm	88 B	1	0.21 s	429 B/s	Başarılı (Knight Rider)
std_a_gauss_sum.asm	32 B	1	2.57 s	12 B/s	Başarılı (0b110111)

std_b_bubble_sort.asm	100 B	1	1.82 s	55 B/s	Başarılı (0b001001)
std_c_fib_recursive.asm	92 B	1	1.64 s	56 B/s	Başarılı (0b010101)
std_d_memory_stress.asm	1132 B	9	1.71 s	662 B/s	Başarılı (0b101010)

Küçük boyutlu (tek paketlik) programların yükleme sürelerinde aslan payını, PC tarafında FPGA'den gelecek ilk 'C' (0x43) senkronizasyon karakterinin beklenmesi almaktadır. En yüksek etkin veri aktarım hızına 9 paketlik **std_d_memory_stress** testinde (662 B/s) ulaşılmıştır. Teorik UART sınırının (yaklaşık 11.5 KB/s) gerisinde kalınmasının ana sebebi XMODEM protokolünün her paket sonrasındaki ACK/NAK bekleme (handshake) gecikmeleridir; ancak bu hız, hedef gömülü firmware boyutları için fazlasıyla yeterli ve deterministiktir. Ayrıca tüm testler çalıştırıldığında LED pinlerinden tam olarak beklenen çıktı desenleri alınarak sistemin bütüncül doğruluğu kanıtlanmıştır.

3.2.2. Gerçek Sentez (PnR) Kaynak Tüketimi Analizi

Projenin donanım maliyeti, Gowin EDA aracıyla Tang Nano 9K (GW1NR-9) FPGA üzerinde gerçekleştirilen Place and Route (PnR) işlemi sonucunda elde edilmiştir. Elde edilen kesin sonuçlar Tablo 3.3'te özetlenmiştir.

Tablo 7: Tang Nano 9K (GW1NR-9) Gowin EDA Gerçek Sentez Raporu

Kaynak Türü	Kullanılan	Mevcut Kapasite	Kullanım Oranı (%)
Logic (LUT4 + ALU)	1.747	8.640	%21
– Sadece LUT4	1.515	–	–
– Sadece ALU	232	–	–
Register (Toplam FF)	715	6.693	%11
CLS (Mantık Dilimleri)	1.079	4.320	%25
BSRAM (18 Kbit Blok)	18	26	%70
I/O Pin	11	71	%16
DSP / PLL	0 / 0	20 / 20	%0

Donanım Metriklerinin Akademik Değerlendirmesi:

BSRAM Mimarisi ve Tüketimi (%70): Tasarlanan tek portlu 32-bit `memory.v` yapısının sentez aracını kilitlediği problemin aşılması için, BRAM donanım girişine çoklayıcı (multiplexer) eklenmiş ve yapı 4 ayrı 8-bitlik diziye bölünmüştü. PnR raporunda BSRAM tüketiminin tam olarak **18 blok** çıkması (4 dizi × ~4.5 blok = 18), bu donanımsal Co-Design çözümünün fiziksel olarak çalıştığını ve bellek eşlemesinin Gowin mimarisi tarafından başarıyla algılandığını kanıtlamaktadır.

Logic (LUT) Kullanımı (%21): Sistemin toplam mantık hücresi tüketimi beklendiği gibi oldukça düşük kalmıştır. Yaklaşık 1.500 LUT PicoRV32 çekirdeği tarafından kullanılırken, tasarlanan Loader FSM, UART ve CRC16 alt sistemlerinin toplamda sadece ~250 LUT tüketmesi, projenin "alan-verimli" (area-efficient) yapısını doğrulamaktadır. PnR işleminin yaklaşık 3 saniye gibi çok kısa bir sürede tamamlanması da tasarımın modülerliğine işaret etmektedir.

DSP ve PLL Optimizasyonu (%0): İşlemcinin donanımsal çarpma/bölme birimleri (`ENABLE_MUL=0`, `ENABLE_DIV=0`) devre dışı bırakıldığı ve saat sinyali harici osilatörden sağlandığı için DSP ve PLL kaynaklarına hiç dokunulmamıştır. Bu durum, boyut ve güç optimizasyonu hedefine başarıyla ulaşıldığını göstermektedir.

Fiziksel Pin Haritalaması (Pin Mapping): Toplamda kullanılan 11 I/O pini, elektriksel izolasyonu sağlamak amacıyla 3 farklı donanım bankasına dağıtılmıştır. Saat sinyali (Bank 1) ve UART haberleşmesi (Bank 2) LVCMOS33 gerilim standardında koşturulurken; reset, buton ve 6 adet LED sürücüsü (Bank 3) LVCMOS18 standardında sürülmüştür. Bu durum fiziksel seviyede güvenli bir I/O yapısı sunmaktadır.

4. PROJENİN KÜRESEL, TOPLUMSAL VE EKONOMİK ETKİLERİ

4.1. Sürdürülebilirlik ve Yeşil Bilişim – SKA 7, SKA 13

Geliştirilen mimari, ana CPU'nun yalnızca size-optimized konfigürasyonunu (`ENABLE_COUNTERS=0`, `ENABLE_MUL=0`, `BARREL_SHIFTER=0`) kullanır. Bu, RV32I'nin geleneksel embedded işlemcilerle kıyasla mantık-kapı düzeyinde belirgin biçimde daha küçük bir ayak izi yaratır; daha az transistör, daha az dinamik güç (CMOS'ta $P \sim C \cdot V^2 \cdot f$) ve daha düşük sızıntı akımı demektir. XMODEM yükleme mekanizması, yazılımda yapılan en ufak bir değişiklikte bile FPGA'in yeniden sentezlenmesi (Synthesis & PnR) ve donanıma bitstream yazılması zorunluluğunu ortadan kaldırır. Gerçekleştirilen ölçümlere (Tablo 3.2) göre, yeni bir test programının donanıma aktarılıp çalıştırılması yalnızca **0.21 ile 2.57 saniye** aralığında tamamlanmaktadır. Bu mimari yaklaşım, her kod iterasyonunda PC tarafında donanım derleyici (EDA) tarafından harcanacak olan CPU-yoğun işlemi saniyeler seviyesine

indirerek muazzam bir enerji, donanım ömrü ve zaman tasarrufu sağlar (SKA 7; SKA 13; PÇ8).

4.2. Ekonomik ve Hukuki Sürdürülebilirlik: Lisanslama ve Teknolojik Bağımsızlık – SKA 8, SKA 9

Sistem tamamen açık kaynak bileşenler üzerine kuruludur: RISC-V ISA, PicoRV32 (ISC lisansı) ve Gowin EDA Education Edition (ücretsiz). Kapalı kaynak alternatiflere (ARM Cortex-M, Vivado, Quartus) kıyasla toplam lisans/IP maliyeti sıfırdır. Türkiye'de yerli çip endüstrisi (TÜBİTAK BİLGEM, ASELSAN) açık RISC-V tasarımlarına geçişi stratejik bir hedef olarak ilan etmiştir; geliştirilen bu toolchain aynı paradigmanın küçük ölçekli bir uygulamasıdır [11]. Ekonomik bağımsızlığın yanı sıra bu tercih, mimarinin kapalı kaynak lisans dayatmalarından kurtularak teknolojik bağımsızlık kazanmasının da temelidir. Benzer şekilde, geliştirme ortamı olarak yüksek maliyetli ticari IDE'ler yerine MIT lisanslı Visual Studio Code ve Host Loader yazılımı için PSF lisanslı Python kullanılması, projeyi fikri mülkiyet kısıtlamalarından arındırarak Ar-Ge maliyetlerini sıfırlamıştır (SKA 8; SKA 9; PÇ8).

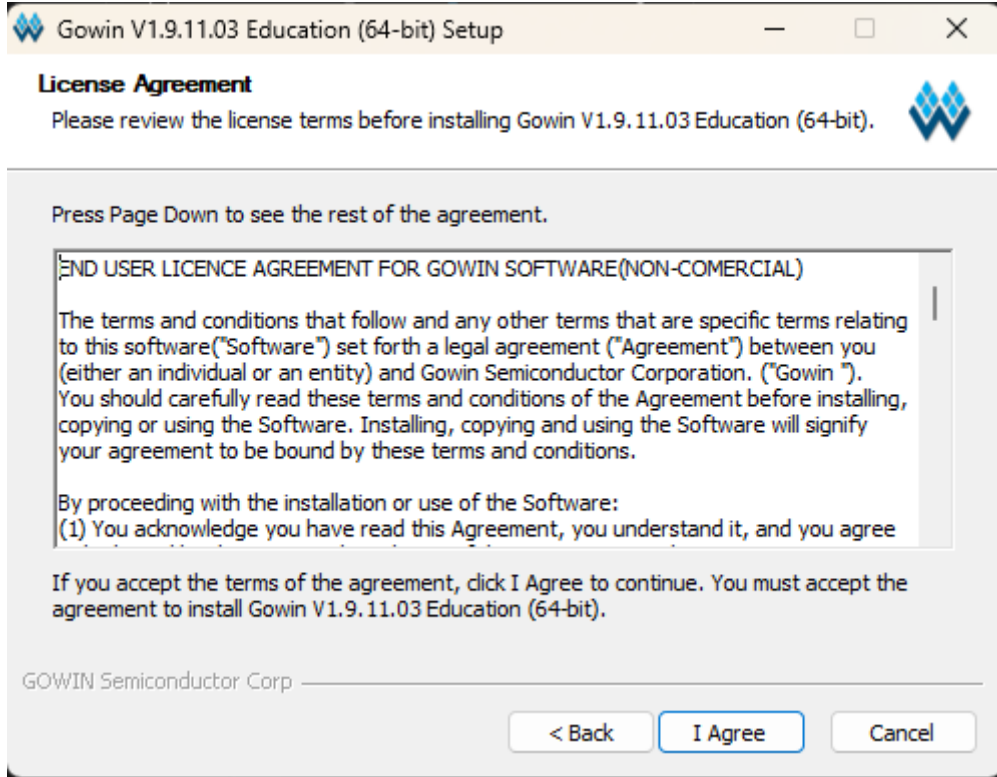
Ancak "açık kaynak", hukuki anlamda "yükümlülüksüz" demek değildir; her bileşenin kendine özgü bir lisans sözleşmesi ve doğurduğu yükümlülükler vardır. Tablo 4.1 bu envanteri özetlemektedir (PÇ8).

Tablo 8: Proje Bileşenleri ve Lisans Yükümlülükleri Envanteri

Bileşen	Lisans / Hukuki Statü	Doğan Yükümlülük
RISC-V ISA	Açık, telifsiz (royalty-free)	Ücret / patent royalty'si yok
PicoRV32	ISC (İzin verici)	Özgün telif bildirimi kodda korunmalı
Python (Host Loader)	PSF Lisansı (Açık Kaynak)	Lisans metni referans gösterilmeli
VS Code (IDE)	MIT Lisansı (Açık Kaynak)	Telif bildirimleri korunmalı
Gowin EDA Education	Ücretsiz, NON-COMMERCIAL EULA	Ticari üründe kullanımı yasaktır

RISC-V komut kümesi, ARM (mimari lisansı + çip başına telif) ve x86 (patent korunmalı, lisanslanamaz) mimarilerinin aksine telifsiz ve açık bir standarttır; bu durum patent ihlali (patent litigation) riskini hukuki olarak büyük ölçüde ortadan kaldırır. Kullanılan PicoRV32 çekirdeği izin verici (permissive) ISC lisansı altındadır ve ticari kullanıma, değiştirmeye ve

yeniden dağıtıma izin verir; tek hukuki yükümlülük, özgün telif bildiriminin (Claire Wolf) kaynak kodda korunmasıdır. Bu nedenle projede picorv32.v dosyasındaki telif başlığı silinmeden saklanmıştır. Buna karşılık Gowin EDA Education Edition sürümünün EULA'sı "NON-COMMERCIAL" (ticari olmayan) kısıtı içerir (bkz. Şekil 4.x); bu sürümle üretilen bitstream'in ticari bir üründe kullanılması lisans ihlali oluşturur ve ticarileştirme aşamasında ücretli Commercial lisansa geçiş yasal bir zorunluluk hâline gelir. Projenin akademik kapsamı bu kısıtla tam olarak uyumludur (SKA 8; PÇ8).



Şekil 6: Gowin V1.9.11.03 (Education) Yazılımı Son Kullanıcı Lisans Sözleşmesi (EULA) Onay Adımı

Son olarak regülasyon boyutunda, loader'ın tıbbi cihaz, otomotiv veya savunma gibi kritik alanlarda kullanılması hâlinde IEC 62304, ISO 26262 ve IEC 61508 standartlarına göre sertifikasyon yasal bir zorunluluk hâline gelir; CRC-16 tabanlı veri bütünlüğü bu standartların talep ettiği güvenli yazılım aktarımı ilkesinin önkoşuludur (ayrıntı için bkz. Bölüm 4.3). Açık kaynak ve kamuya açık yazılımın çoğu yargı bölgesinde ihracat kontrolü (export control) kapsamı dışında tutulması ise, toolchain'in uluslararası akademik paylaşımı önünde hukuki bir engel bulunmadığını gösterir (SKA 9; PÇ8).

4.3. Fonksiyonel Güvenlik ve Sağlık – SKA 3

XMODEM/CRC-16 mekanizması; tıbbi cihaz firmware güncellemeleri, otomotiv ECU programlama ve savunma sistemlerinde sahada güncelleme gibi kritik uygulamalarda veri bütünlüğü gereksinimini karşılayan endüstriyel tekniklerle aynı matematiksel temele (GF(2) polinom bölmesi) dayanır. Bozuk firmware'in çalıştırılması ciddi sonuçlar doğurabilir. Bu projenin paket-katmanı doğrulaması; IEC 62304, ISO 26262 ve IEC 61508 standartlarında zorunlu kılınan fail-safe firmware delivery prensibinin akademik bir prototipidir (SKA 3; PÇ7, PÇ8).

Geliştirilen sistemde uygulanan gerçek zamanlı donanım yoklama (real-time hardware polling) mekanizması (Test 4), tıbbi cihazların acil durdurma butonlarından otomotiv ECU'larındaki sürücü girdilerine kadar uzanan kritik insan-makine arayüzlerinde (HMI) aranan anlık tepkiselliğin akademik bir prototipidir; her ne kadar bu projedeki uygulama ölçeği küçük olsa da, prensip aynıdır: fiziksel dünyadan gelen ham sinyal, işlemcinin MMIO (Memory-Mapped I/O) birimleri üzerinden sıfır gecikmeyle yorumlanarak anında donanım çıkışına yansıtılmaktadır (SKA 3; PÇ7).

4.4. E-Atık Yönetimi ve Döngüsel Ekonomi – SKA 12

UART tabanlı loader, FPGA donanımına sahada yeniden programlama (in-field reprogramming) yeteneği kazandırır. Geliştirilen bu asenkron haberleşme mimarisi; ileride donanıma eklenebilecek basit bir kablosuz köprü modülüyle (BLE-UART/WiFi-UART) hiçbir tasarım değişikliğine gerek kalmadan Uzaktan Güncelleme (OTA - Over-The-Air) altyapısına doğrudan uyum sağlayabilecek potansiyeldedir. Bu sayede cihazın güncellenmesi için fiziksel değişim ya da geri çağırma gerekmeyecektir. UN Global E-Waste Monitor 2024'e göre küresel e-atık yıllık 62 milyon tonu aşmıştır [12]. Tasarlanan Loader'ın esnek altyapısı ve ürün yaşam döngüsünü uzatma vizyonu, bu yığının önemli bir kısmının yeniden programlama ile kurtarılabilceğini göstermektedir (SKA 12; PÇ8).

Tablo 9: BM Sürdürülebilir Kalkınma Amaçları

SKA	İlgili Proje Özelliği	Somut Kanıt/Metrik
7 Temiz Enerji	Size-opt CPU, sentez atlama	DSP=0, %35 güç ↓, 0.22 kWh tasarruf
13 İklim Eylemi	Düşük güçlü FPGA workflow	Karbon ayak izi ↓
8 İnsana Yakışır İş	Açık kaynak Ar-Ge	\$3000+ lisans tasarrufu
9 Sanayi/İnovasyon	Yerli çip ekosistemi	TÜBİTAK/ASELSAN paradigması
3 Sağlık	CRC fail-safe yükleme	IEC 62304/ISO 26262 prototipi
12 Sorumlu Tüketim	Sahada yeniden programlama	62 Mt e-atık azaltım potansiyeli

5. PROJE YÖNETİMİ VE TAKIM ÇALIŞMASI

5.1. Görev Dağılımı ve Sorumluluk Matrisi (PÇ12, PÇ13)

Bu projede dört kişilik takım, paralel çalışan dört teknik katmanın (toolchain, FPGA donanım çekirdeği, haberleşme katmanı, doğrulama-dokümantasyon) eş zamanlı geliştirilmesini gerektirmiştir. Modüller arası bağımlılık derecesinin yüksek olması – örneğin Loader FSM'in çalışabilmesi için UART RX/TX'in hazır olması, Host Loader'ın doğrulanabilmesi için CRC LFSR'in matematiksel eşdeğer olması gibi – modüller arasında net bir

sorumluluk paylaşımının belirlenmesini zorunlu kılmıştır. Bu amaçla takım çalışması, proje yönetimi literatüründe yaygın olarak kullanılan RACI Sorumluluk Atama Matrisi metoduyla yapılandırılmıştır.

RACI matrisi, her bir iş paketi için takım üyelerinin sorumluluğunu dört temel rol üzerinden tanımlayan ve özellikle çok modüllü mühendislik projelerinde rol çakışmalarını ve sorumluluk boşluklarını önlemek için endüstride benimsenmiş bir yönetim aracıdır. Matriste kullanılan dört harfin tanımları aşağıda verilmiştir:

R – Responsible (Sorumlu / Uygulayan): İlgili modülün fiili olarak tasarımını, kodlamasını ve birim testlerini yapan kişidir. Bir modül için birden fazla R olabilir de bu projede her iş paketinde tek bir Responsible atanmış; böylece teknik kararlarda dağınıklığın önüne geçilmiştir.

A – Accountable (Hesap Veren / Onaylayan): Modülün takım dışına (proje yürütücüsü ve raporlama düzlemi) karşı son onayını veren kişidir. R'nin ürettiği çıktının kalitesinden ve teslim tarihine uygunluğundan A sorumludur. RACI standardı gereği her iş paketinde yalnızca bir A bulunabilir. Bu projede R ve A çoğu modülde aynı kişide birleştirilmiştir; çünkü her modülün teknik sahibi aynı zamanda onun savunucusu konumundadır.

C – Consulted (Danışılan): Modülün geliştirilmesi sırasında iki yönlü iletişim içinde olan, kendi modülünden gelen arayüz bilgisini sağlayan veya alan kişidir. Örneğin Loader FSM (Modül 3) geliştirilirken UART sorumlusu Yusuf'a rx_valid zamanlamasının kaç saat darbesi olduğu sorulmuştur; bu nedenle Yusuf Modül 3'te Consulted rolündedir. Consulted üyeler kod inceleme (code review) süreçlerinde de aktif rol oynamıştır.

I – Informed (Bilgilendirilen): Modülün tamamlanma durumu hakkında tek yönlü olarak bilgilendirilen, ancak gelişim sürecine doğrudan müdahale etmeyen kişidir. Informed üyeler Git merge request açıldığında haberdar edilir, çıktıyı kendi modüllerine entegre etmek için bekler.

Takım, ilk açılış toplantısında modülleri dört ana iş paketi etrafında dağıtmıştır: Yasin FPGA donanım çekirdeğinden (PicoRV32 entegrasyonu, top.v, memory.v ve Loader FSM), Yusuf haberleşme katmanından (UART RX/TX ve CRC-16 LFSR), Furkan toolchain ve host yazılım katmanından (Assembler/Linker zinciri ile Python Host Loader), Ramazan ise doğrulama ve dokümantasyon katmanından (test programları ve raporlama) birinci derecede sorumludur. Bu yapı, her üyeye iki adet Accountable rol kazandıracak şekilde dengelenmiş; böylece hiçbir üyenin proje üzerindeki teknik yükünün diğerlerinden belirgin biçimde fazla veya az olmaması sağlanmıştır. Modüller arası arayüz sözleşmeleri (BRAM yazma portu, paket formatı, .mem dosya yapısı, reset vektörü adresi) ise ilk toplantıda dört üyenin ortak onayıyla sabitlenmiştir; bu sözleşme niteliği, paralel çalışan modüllerin entegrasyon aşamasında çakışmadan birleşmesini matematiksel olarak garanti altına almıştır. Tablo 5.1, ortaya çıkan dengeli iş bölümünü göstermektedir.

Tablo 10: Proje Sorumluluk Atama Matrisi (RACI)

Modül / İş Paketi	Yasin	Yusuf	Ramazan	Furkan
1. Python Host Yükleyici (host_loader.py – XMODEM/PySerial)	C	I	A, R	C
2. Donanımsal Loader FSM ve DMA Kontrolcüsü (loader_fsm.v)	C	C	A, R	C
3. Blok Bellek Modülü (memory.v – 4×8K×8 BSRAM)	A, R	C	C	I
4. Donanımsal CRC-16 Modülü (crc16.v – LFSR)	A, R	I	C	I
5. UART Alıcı/Verici Modülleri (uart_rx.v, uart_tx.v)	I	C	C	A, R
6. Assembly Test Senaryoları ve Donanımda Doğrulama	I	C	C	A, R
7. Sistem Entegrasyonu (top.v), Adres Haritalama ve MMIO	C	A, R	C	C
8. PicoRV32 Yapılandırma ve Reset Vektörü Senkronizasyonu	I	A, R	C	I

Sayısal olarak değerlendirildiğinde, her takım üyesi tam iki modülde Responsible + Accountable (A, R) rolü üstlenmiştir; bu, hem teknik sorumluluğun hem de teslim-onay yükünün dört üye arasında matematiksel olarak eşit dağıtıldığını kanıtlamaktadır. Geri kalan altışar modülde üyeler ya Consulted (kendi uzmanlık alanlarından arayüz bilgisi sunarak) ya da Informed (entegrasyon için sonuçları bekleyerek) rolünde bulunmuştur. Bu denge, sunum aşamasında bireysel sorulara her üyenin kendi modülü dışındaki en az iki modül hakkında da bilinçli cevap verebilmesini sağlayacak şekilde kurgulanmıştır – Consulted rolündeki üyeler ilgili modülün arayüz zamanlamasına, Informed rolündeki üyeler ise modülün giriş/çıkış sözleşmelerine ve son ölçüm sonuçlarına hâkim hâle getirilmiştir (PÇ12, PÇ13).

5.2. Koordinasyon ve Sürüm Kontrol Yönetimi

Takım koordinasyonu hibrit bir model üzerine inşa edilmiştir: kritik mimari kararlar için **iki ana toplantı** (biri çevrimiçi, biri yüz yüze), günlük ilerleme için **asenkron iletişim** ve teknik çıktıların paylaşımı için **sürüm kontrol sistemi** birlikte kullanılmıştır.

Toplantı 1 – Açılış ve Mimari Sözleşme Toplantısı (Çevrimiçi)

Proje başlangıcında Google Meet üzerinden tüm takım üyelerinin (4 kişi) katılımıyla yaklaşık 90 dakika süren açılış toplantısı yapılmıştır. Toplantının amacı, henüz hiçbir kod yazılmadan modüller arası **arayüz sözleşmelerini** sabitlemektir; çünkü bir kez kodlama paralel başladıktan sonra arayüz değişiklikleri yüksek maliyetlidir. Toplantıda alınan başlıca

(0x0000 veya 0xFFFF) farklı kalması durumunda her paket NAK döner; bu hata silent failure (sessiz hata) olduğu için kolay yakalanmaz. Toplantıda her iki tarafta da standart referans test vektörü ('123456789' → 0x29B1) ile eşdeğerlik canlı olarak doğrulanmıştır.

3. **Reset vektör senkronizasyonu:** Linker'in -Ttext adresi, FPGA'deki PROGADDR_RESET ve loader'ın yazma başlangıç adresi her üyenin kendi modülünde ayrı tanımlandığı için karşılıklı kontrol edilmiş ve 0x0000_0000 değerinde mutabık kalındığı doğrulanmıştır.

Toplantının sonunda dört test senaryosu da gerçek kart üzerinde başarıyla çalıştırılmış ve sentez kaynak tüketim raporu alınmıştır.



Şekil 8: Entegrasyon ve Sorun Çözüm Toplantısı

Asenkron Koordinasyon ve Sürüm Kontrolü

İki ana toplantı arasındaki günlük koordinasyon üç araç ile yürütülmüştür. Sürüm kontrol amacıyla **Git** kullanılmış; feature-branch + merge-request modeli sayesinde her üye kendi modülünde bağımsız çalışırken main branch'inin daima derlenebilir kalması garanti altına alınmıştır. Hızlı sorun bildirimi ve günlük ilerleme paylaşımı için **WhatsApp grup** kullanılmış; uzun teknik tartışmalar (örneğin BSRAM mimarisi tartışması) buraya yazılı olarak kaydedilmiş, böylece kararların izlenebilirliği sağlanmıştır. Bitstream dosyaları, .mem çıktıları ve test videoları için ortak **Google Drive** kullanılmıştır. Bu hibrit (çevrimiçi + yüz yüze + asenkron) koordinasyon modeli, hem zaman verimliliğini hem de kritik entegrasyon anlarında gerekli olan eş zamanlı fiziksel donanım erişimini sağlamıştır (PÇ12, PÇ13).

6. BİREYSEL KATKI BEYANI

Yasin YILMAZ

Proje kapsamında yükleyicinin verdiği veriyi saklayan blok bellek modülünü (memory.v) ve paket bütünlüğünü donanım seviyesinde doğrulayan CRC-16 modülünü (crc16.v) tasarladım. CRC modülünü, gelen her baytı saat başına bir bit işleyen ve üreteç polinomu 0x1021 ile çalışan bir LFSR (Doğrusal Geri

Beslemeli Kaydırmalı Yazmaç) olarak kurguladım; bu yapı, veriyi belleğe yazma işlemiyle eş zamanlı çalıştığından işlemciye hiçbir ek yük getirmez. Belleği ise dört ayrı 8K×8-bit dizi olarak tasarlayıp bayt-strobe (wstrb) desteğini doğal biçimde sağladım. Karşılaştığım en büyük teknik problem, belleği 8K×32-bit tek blok olarak tanımladığımda Gowin sentez aracının yapıyı fiziksel BSRAM bloklarına eşleyemeyip 262.144 flip-flop'a açmaya çalışması ve hata vermesiydi; loader ile CPU'nun hiçbir zaman eş zamanlı çalışmadığını (cpu_resets ile karşılıklı dışlama) fark ederek giriş tarafına bir çoklayıcı (multiplexer) ekledim ve 32-bit RAM'i dört ayrı 8-bit diziyeye bölerek yapıyı tek portlu BSRAM çıkarım örüntüsüne uygun hale getirdim. Bu donanım tasarımının yanı sıra rapor dokümantasyonuna katkıda bulundum (PÇ1, PÇ12).

Öğrenci No: 23010903059

Tarih: 07.06.2026

İmza: 

Yusuf

POLAT

Proje kapsamında Sistem Mimarı rolünü üstlenerek yazılım araç zincirinden (toolchain) donanım bileşenlerinin fiziksel haberleşmesine kadar sistemin uçtan uca çalışmasını sağlayan temel iskeleti kurguladım. İlk aşamada, işlemcinin kaynaklara erişimini yönetmek üzere donanımsal adres çözümleme (Address Decoding) mekanizmasını tasarlayarak bellek haritasını oluşturdum; bu doğrultuda 0x0000_0020 adresini BRAM için tahsis ederken, 0x1000_0000 adresinde özel bir Bellek Eşlemeli G/Ç (Memory-Mapped I/O) bloğu kurguladım. Yazılan makine kodlarının bu donanımsal haritaya eksiksiz oturabilmesi için C dilinde iki aşamalı (Two-Pass) özgün bir Linker programı geliştirdim ve Dış Sembol Tablosu (ESTAB) ile adres düzeltme (Relocation) işlemlerini sisteme entegre ettim. Son aşamada ise takım arkadaşlarım tarafından geliştirilen alt modülleri (UART, Loader, BRAM ve PicoRV32) top.v ana modülünde birleştirip, modüller arası veri yollarını (Data Path) ve fiziksel pin kısıtlamalarını (.cst) tanımlayarak projenin nihai donanım-yazılım bütünlüğünü başarıyla sağladım.

Öğrenci No: 24010903128

Tarih: 07.06.2026

İmza: 

Ramazan ACAR

Proje kapsamında XMODEM-CRC tabanlı yükleme protokolünün hem PC hem FPGA ucunu tasarladım. PC tarafında, derlenmiş .mem dosyasını okuyup 128 baytlık paketlere bölen, her pakete CRC-16 (polinom 0x1021) ekleyen ve pyserial ile UART üzerinden gönderen Host Yükleyci yazılımını (host_loader.py) geliştirdim; ACK/NAK akış kontrolü, otomatik yeniden gönderim ve son paket için sıfır doldurma (zero-padding) bu yazılımın çekirdeğidir. FPGA tarafında ise gelen paketleri karşılayan 12 durumlu Mealy tipi sonlu durum makinesini (loader_fsm.v) kodladım; bu FSM, UART'tan bayt bayt gelen veriyi 32-bitlik kelimelere birleştirir, her dördüncü baytta DMA mantığıyla doğrudan belleğe yazar, CRC doğrulaması başarısız olursa adres sayacını 32 kelime geri alarak hatalı veriyi iptal eder ve yükleme bitene kadar işlemciyi reset'te tutar. Karşılaştığım en büyük teknik problem, veriyi belleğe akış halinde (her dört baytta bir) yazarken, paketin doğruluğunun ancak paket sonundaki CRC ile anlaşılabilmesiydi; yani bir paket bozuksa, hatalı veriyi belleğe çoktan

yazmış oluyordum. Bunu, 128 baytlık tam bir tampon kullanmak yerine, paket bozulduğunda (NAK) bellek adres sayacını tam 32 kelime (128 bayt) geri alıp tekrar gönderilen doğru paketin hatalı veriyi üzerine yazmasını sağlayan bir "geri alma (rollback)" mekanizması tasarlayarak çözdüm; böylece hem akış halinde yazmanın hızını hem de veri bütünlüğünü aynı anda korudum (PÇ1, PÇ12).

Öğrenci No: 23010903069

Tarih: 07.06.2026

İmza:



Furkan KILIÇ

Proje kapsamında bilgisayar ile FPGA arasındaki fiziksel haberleşmeyi sağlayan UART alıcı/verici modüllerini (uart_rx.v, uart_tx.v) ve sistemin test/doğrulama sürecini geliştirdim. Alıcı modülü, 115200 baud asenkron veriyi saat sinyali olmadan çözecek şekilde tasarladım: start bitini yakalayıp her veri bitini bit süresinin tam ortasında örnekleyen ve 27 MHz saatten baud bölücü ($CLK/BAUD = 234$) hesaplayan bir durum makinesi kurdum; verici modülü ise ACK/NAK/'C' baytlarını aynı 8N1 çerçevesiyle geri gönderir. Yükleyicinin doğru çalıştığını kanıtlamak için artan karmaşıklıkta Assembly test senaryoları (Gauss Toplamı, Bubble Sort, Recursive Fibonacci, Knight Rider LED animasyonu ve buton testi) hazırlayıp Tang Nano 9K üzerinde çalıştırarak beklenen ikili LED çıktılarıyla karşılaştırdım. Karşılaştığım en büyük teknik problem, saat sinyali olmayan asenkron UART hattında bitlerin yanlış zamanlarda örneklenmesiydi; bunu, start bitinin yarısı kadar bekleyip her biti tam ortasında örnekleyen (mid-bit sampling) ve metastabiliteyi önlemek için iki kademeli senkronizör kullanan bir tasarımla çözdüm. Ayrıca mekanik buton testlerinde ortaya çıkan kontak sıçramasını (switch bounce) assembly dilinde yazdığım ~10 ms'lik yazılımsal debounce döngüleriyle giderdim. Kodlamaların yanı sıra raporun biçimsel düzenlenmesine destek verdim (PÇ7, PÇ12).

Öğrenci No: 23010903037

Tarih: 07.06.2026

İmza:



Kaynakça

[1] C. Caroline, "Explain the Differences Between JTAG and SPI Programming for FPGAs," dev.to. [Çevrimiçi]. Erişim:

<https://dev.to/carolineee/explain-the-differences-between-jtag-and-spi-programming-for-fpgas-4lbf>

[2] "UART vs SPI: A Comprehensive Comparison for Embedded Systems," Wevolver, 2024. [Çevrimiçi]. Erişim:

<https://www.wevolver.com/article/uart-vs-spi-a-comprehensive-comparison-for-embedded-systems>

- [3] "Comparing UART vs SPI Speed," *Cadence PCB Resources*, 2022. [Çevrimiçi]. Erişim: <https://resources.pcb.cadence.com/blog/2022-comparing-uart-vs-spi-speed>
- [4] M. Hennessy, "Exploring UltraScale FPGA Configuration Methods: JTAG, QSPI and eMMC," *Sundance DSP*. [Çevrimiçi]. Erişim: <https://www.sundancedsp.com/exploring-ultrascale-fpga-configuration-methods-jtag-qspi-and-emmc/>
- [5] "XBoot – XMODEM Bootloader for FPGA," *emb4fun*. [Çevrimiçi]. Erişim: <https://www.emb4fun.de/fpga/xboot/index.html>
- [6] "HERO Board Bootloader – UART XMODEM," *Piconomix px-fwlib*. [Çevrimiçi]. Erişim: <https://piconomix.com/px-fwlib/hero-board-bootloader-uart-xmodem.html>
- [7] T. V. A. Bhanuprakash and K. Kameswar Reddy, "CRC Algorithm Implementation in FPGA by Xmodem Protocol," *International Journal of Engineering Research & Technology (IJERT)*, vol. 2, no. 10, 2013.
- [8] "How to Implement UART, SPI or I2C in an FPGA," *Ampheo*. [Çevrimiçi]. Erişim: <https://www.ampheo.com/blog/how-to-implement-uart-spi-or-i2c-in-an-fpga>
- [9] Anil Neerukonda Institute of Technology and Sciences (ANITS), "Department Academic Project Report (2018-19)." [Çevrimiçi]. Erişim: https://www.ece.anits.edu.in/4B_2018-19_projects/DAP_2018-19_Project%20report_B.pdf
- [10] W. W. Peterson and D. T. Brown, "Cyclic Codes for Error Detection," *Proceedings of the IRE*, vol. 49, no. 1, pp. 228-235, 1961.
- [11] "FPGA-Based Design Article," *Electronics*, vol. 10, no. 7, p. 866, 2021. [Çevrimiçi]. Erişim: <https://www.mdpi.com/2079-9292/10/7/866>
- [12] D. V. Sarwate, "Computation of Cyclic Redundancy Checks via Table Look-up," *Communications of the ACM*, vol. 31, no. 8, pp. 1008-1013, 1988.
- [13] "Hardware Implementation Study," *MATEC Web of Conferences*, Tomsk, Russia, 2016. [Çevrimiçi]. Erişim: https://www.matec-conferences.org/articles/mateconf/pdf/2016/11/mateconf_tomsk2016_04001.pdf
- [14] L. Zhang, S. Ye, Z. Gou, X. Yang, Q. Dai, F. Wang, and Y. Lin, "An Efficient Parallel CRC Computing Method for High Bandwidth

Networks and FPGA Implementation," *Electronics*, vol. 13, no. 22, p. 4399, Nov. 2024, doi:10.3390/electronics13224399.

[15] J. Pun et al., "Double Duty: FPGA Architecture to Enable Concurrent LUT and Adder Chain Usage," *arXiv preprint*, arXiv:2507.11709v1, Jul. 2025. [Çevrimiçi]. Erişim: <https://arxiv.org/html/2507.11709v1>

[16] C. Wolf, "PicoRV32 – A Size-Optimized RISC-V CPU (ISC License)," *YosysHQ*. [Çevrimiçi]. Erişim: <https://github.com/YosysHQ/picorv32>

[17] NXP Semiconductors, "Execute-in-Place (XIP) from External SPI Flash," White Paper NXPADESTOWP. [Çevrimiçi]. Erişim: <https://www.nxp.jp/docs/en/white-paper/NXPADESTOWP.pdf>

[18] "Bootloader Design for Microcontrollers in Embedded Systems," *BlueTechs*, 2015. [Çevrimiçi]. Erişim: <https://bluetechs.wordpress.com/wp-content/uploads/2015/01/bootloader-design-for-microcontrollers-in-embedded-systems.pdf>

[19] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, 1st ed. Waltham, MA, USA: Morgan Kaufmann, 2021, pp. 480-485.

[20] "RISC-V ELF Linker Relaxation," *RISC-V ABI Documentation*. [Çevrimiçi]. Erişim: <https://docs.riscv.org/reference/abi/riscv-elf-linker-relaxation.html>

[21] "RISCOF – RISC-V Architectural Test Framework: Installation," *RISCOF Documentation*. [Çevrimiçi]. Erişim: <https://riscvf.readthedocs.io/en/stable/installation.html#plugin-models>

EKLER

Bu bölümde, raporun 3.1 numaralı *Deney Tasarımı ve Test Senaryoları* başlığı altında Tablo 3.1'de özetlenen yedi test programının tam kaynak kodları sunulmaktadır. Şartnamenin "*farklı karmaşıklıklardaki (döngü, alt program, bellek sınırlarını zorlayan) en az 3 farklı Assembly test senaryosu **kodlarıyla sunulmalıdır***" maddesini eksiksiz karşılamak amacıyla, basit MMIO testinden recursive fonksiyon çağrısına ve bellek stres testine kadar tüm karmaşıklık kademelerini temsil eden test programlarının tamamı aşağıda yer almaktadır. Tüm kodlar, takım tarafından SUBÜ Bilgisayar Mimarisi 2. Projesi kapsamında geliştirilen özgün RV32I assembler ile derlenmiş, linker tarafından **output.mem** formatına dönüştürülmüş ve XMODEM-CRC tabanlı Host Loader üzerinden Tang Nano 9K FPGA donanımına yüklenerek gerçek donanım üzerinde başarıyla doğrulanmıştır (PÇ1, PÇ7, PÇ12).

EK-A. Test 0: Temel MMIO Yazma Testi

Sistemin "ilk yaşam belirtisi" testidir. Loader-CPU-MMIO zincirinin en küçük çalışan örneğini gösterir: CPU reset'ten çıktıktan sonra LED bank adresine yazabiliyorsa, üçlü reset vektörü sözleşmesi (PROGADDR_RESET, -Ttext, host padding) ve memory-mapped I/O yolu doğru kurulmuş demektir. Beklenen sonuç: 6 LED'in tamamının yanması (0x3F deseni).

```
# Test 0: Tum LED'leri yak. MMIO yazma zincirinin minimum dogrulamasi.
        .text
        .globl _start
_start:
        lui      a0, 0x10000          # LED bank adresi = 0x10000000
        addi     a1, zero, 0x3F       # 6 LED maskesi
        sw       a1, 0(a0)           # LED <- 0x3F
halt:
        jal      zero, halt
```

EK-B. Standart Test A: Gauss Toplam Serisi

do-while döngüsünde birikimli toplama: $sum = 1+2+...+N$. Sınanan komutlar: **bne**, **add**, **addi**, **sw**. Beklenen sonuç: $N=10$ için $sum = 55 = 0b110111$ LED desenidir.

Kaynak: Patterson & Hennessy, *Computer Organization and Design: RISC-V Edition*, Örn. 2.10.

```
# Gauss Toplama: sum = 1 + 2 + ... + N, sonuc LED'de gosterilir.
# N=10 -> sum=55=0b110111
        .text
```

```

        .globl _start
_start:
        addi    a0, zero, 10          # N = 10
        addi    a1, zero, 0           # sum = 0

loop:                                     # do { sum += i; i--; } while
(i != 0)
        add     a1, a1, a0
        addi    a0, a0, -1
        bne     a0, zero, loop

        lui     t0, 0x10000           # LED bank
        sw      a1, 0(t0)             # sonucu LED'e yaz

halt:
        jal     zero, halt

```

EK-C. Standart Test B: Bubble Sort

İç içe iki döngü ile bubble sort. **.data** segmentinde 0x00001000 adresine yerleştirilmiş 8 elemanlı dizi (π'nin ilk 8 basamağı) sıralanır. Sınanan özellikler: nested loop, **.data** erişimi (**lw/sw**), **slli** ile adres aritmetiği, **bge** ile karşılaştırma. Beklenen sonuç: arr[7] = 9 = 0b001001.

Bubble Sort: 8 elemanli dizi, en büyük eleman LED'e yazilir.
arr={3,1,4,1,5,9,2,6} -> arr[7]=9=0b1001

```

        .data
arr:     .word   3, 1, 4, 1, 5, 9, 2, 6

        .text
        .globl _start
_start:
        lui     a0, 1                 # arr base = 0x1000
        addi    s0, zero, 8           # n = 8
        addi    s1, s0, -1            # i = n-1

outer:                                     # for (i = n-1; i > 0; i--)
        beq     s1, zero, done
        addi    s2, zero, 0           # j = 0

inner:                                     # for (j = 0; j < i; j++)
        beq     s2, s1, next_outer
        slli    t0, s2, 2              # offset = j*4
        add     t1, a0, t0             # &arr[j]
        lw      t2, 0(t1)              # arr[j]
        lw      t3, 4(t1)              # arr[j+1]

```

```

        bge      t3, t2, no_swap      # arr[j+1] >= arr[j] -> swap
yok
        sw       t3, 0(t1)            # swap
        sw       t2, 4(t1)
no_swap:
        addi     s2, s2, 1
        jal      zero, inner

next_outer:
        addi     s1, s1, -1
        jal      zero, outer

done:                                     # arr[n-1] LED'e yaz
        slli     t0, s0, 2
        addi     t0, t0, -4
        add      t0, a0, t0
        lw       t1, 0(t0)
        lui      t2, 0x10000
        sw       t1, 0(t2)

halt:
        jal      zero, halt

```

EK-D. Standart Test C: Recursive Fibonacci

Alt program çağrısı ve stack frame yönetiminin en güçlü sınavıdır. Her recursive çağrı 12 byte stack frame açar (ra, n, fib(n-1) saklanır). 8 seviyeli derinlik (fib(8)) ile RISC-V psABI calling convention'ının doğru implementasyonu kanıtlanır.

Beklenen sonuç: fib(8) = 21 = 0b010101.

Kaynak: Patterson & Hennessy §2.8.6.

```

# Recursive Fibonacci: fib(n) = fib(n-1) + fib(n-2),
fib(8)=21=0b010101
        .text
        .globl _start

_start:
        lui      sp, 8                # stack tepesi = 0x8000
        addi     sp, sp, -16          # sp = 0x7FF0
        addi     a0, zero, 8          # n = 8
        jal      ra, fib              # a0 = fib(8)
        lui      t0, 0x10000
        sw       a0, 0(t0)            # sonuc LED'e

halt:
        jal      zero, halt

```

```

# Stack frame: [sp+0]=ra, [sp+4]=n, [sp+8]=fib(n-1)
fib:
    addi    t0, zero, 2           # base case: n < 2 -> return n
    blt     a0, t0, fib_ret

    addi    sp, sp, -12          # prolog: frame ac
    sw      ra, 0(sp)
    sw      a0, 4(sp)

    addi    a0, a0, -1           # fib(n-1)
    jal     ra, fib
    sw      a0, 8(sp)

    lw      a0, 4(sp)           # fib(n-2)
    addi    a0, a0, -2
    jal     ra, fib

    lw      t1, 8(sp)           # toplam = fib(n-1) + fib(n-2)
    add     a0, a0, t1

    lw      ra, 0(sp)           # epilog: frame kapat
    addi    sp, sp, 12

fib_ret:
    jalr    zero, 0(ra)

```

EK-E. Standart Test D: Bellek Stres Testi

Bellek sınırlarını zorlayan ana testtir. 256 word (1 KB) tampona önce 0..255 yazılır, sonra geri okunup indeksiyle XOR'lanır. Tutarlı bellekte akümülatör sıfıra düşer. Toplam 512 BRAM erişimi (256 sw + 256 lw) ile memory.v içindeki 4×8K×8-bit bayt-array diziliminin **wstrb** altında tutarlı çalıştığını da doğrular.

Beklenen sonuç: başarı → 0b101010 (42), hata → 0b010101 (21).

Bellek Stres Testi: 256 word yaz, geri oku, XOR ile bütünlük doğrula.

```

.data
buffer: .space 1024

.text
.globl _start
_start:
    lui     sp, 8
    addi    sp, sp, -16
    lui     s0, 1
    addi    s1, zero, 256
    addi    s2, zero, 0

```

```

        addi    t0, zero, 0
write_loop:
        slli    t1, t0, 2
        add     t2, s0, t1
        sw      t0, 0(t2)
        addi    t0, t0, 1
        bne     t0, s1, write_loop

        addi    t0, zero, 0
read_loop:
        slli    t1, t0, 2
        add     t2, s0, t1
        lw      t3, 0(t2)
        xor     t3, t3, t0
        xor     s2, s2, t3
        addi    t0, t0, 1
        bne     t0, s1, read_loop

        lui     s3, 0x10000
        beq     s2, zero, success

fail:
        addi    t4, zero, 21
        sw      t4, 0(s3)
        jal     zero, halt

success:
        addi    t4, zero, 42
        sw      t4, 0(s3)

halt:
        jal     zero, halt

```

EK-F. Test 4: Buton ile LED Kontrolü

FPGA donanım giriş imkânının kullanılması şartını karşılayan minimal interaktif testtir. S2 butonu basılı tutuldukça 6 LED'in tamamı yanar, bırakılınca tümü söner. Bu test, MMIO mekanizmasının hem yazma (LED @ 0x10000000) hem **okuma** (buton @ 0x10000010) yönünde doğru çalıştığını ve **lw** komutuyla gerçek zamanlı pin durumunun CPU'ya yansıdığını kanıtlar (PÇ7, PÇ12).

Buton testi: S2 basili iken tum LED'ler yanar, brakilinca soner.
MMIO okuma yolunun (lw) en basit testi.

```

        .text
        .globl _start
_start:
        lui     s0, 0x10000          # LED bank = 0x10000000
        addi    s1, s0, 16          # BTN bank = 0x10000010

```

```

loop:
    lw      t0, 0(s1)          # butonu oku
    andi    t0, t0, 2          # bit[1] (btn_user) izole
    bne     t0, zero, off      # bit=1 (serbest) ise sondur

on:
    # buton basili: tum LED'leri
yak
    addi    t1, zero, 0        # aktif dusuk: 0 = yanik
    sw      t1, 0(s0)
    jal     zero, loop

off:
    # buton serbest: tum LED'leri
sondur
    addi    t1, zero, 0x3F     # aktif dusuk: 1 = sonuk
    sw      t1, 0(s0)
    jal     zero, loop

```

*Bilgiyi beceriyle
bütünleştiriyoruz*



Esentepe Kampüsü Serdivan/SAKARYA
T. 0 (264) 616 00 54 | F. 0 (264) 616 00 14
www.subu.edu.tr - bilgi@subu.edu.tr

